

DP-601: Implement a lakehouse with Microsoft Fabric

Introduction

<https://learn.microsoft.com/en-us/training/courses/dp-601t00>





Mike Lively

Founder QuantumAI
Trainer, IT Evangelist,
Developer
@mikelively-quantumai



AWS Architect
Google Architect (Gemini)
18 Azure Cloud Certs

About Me

- Father of Nine, USAF
- Working on a PhD in GenAI
- Teaching AI for Johns Hopkins & GK
- Fastlane, QA
- Avid Hacker & Prompt Engineer
- 40 years programming, Keynote
- Flautist
- YMCA Gym Rat

- PhD “ABD”

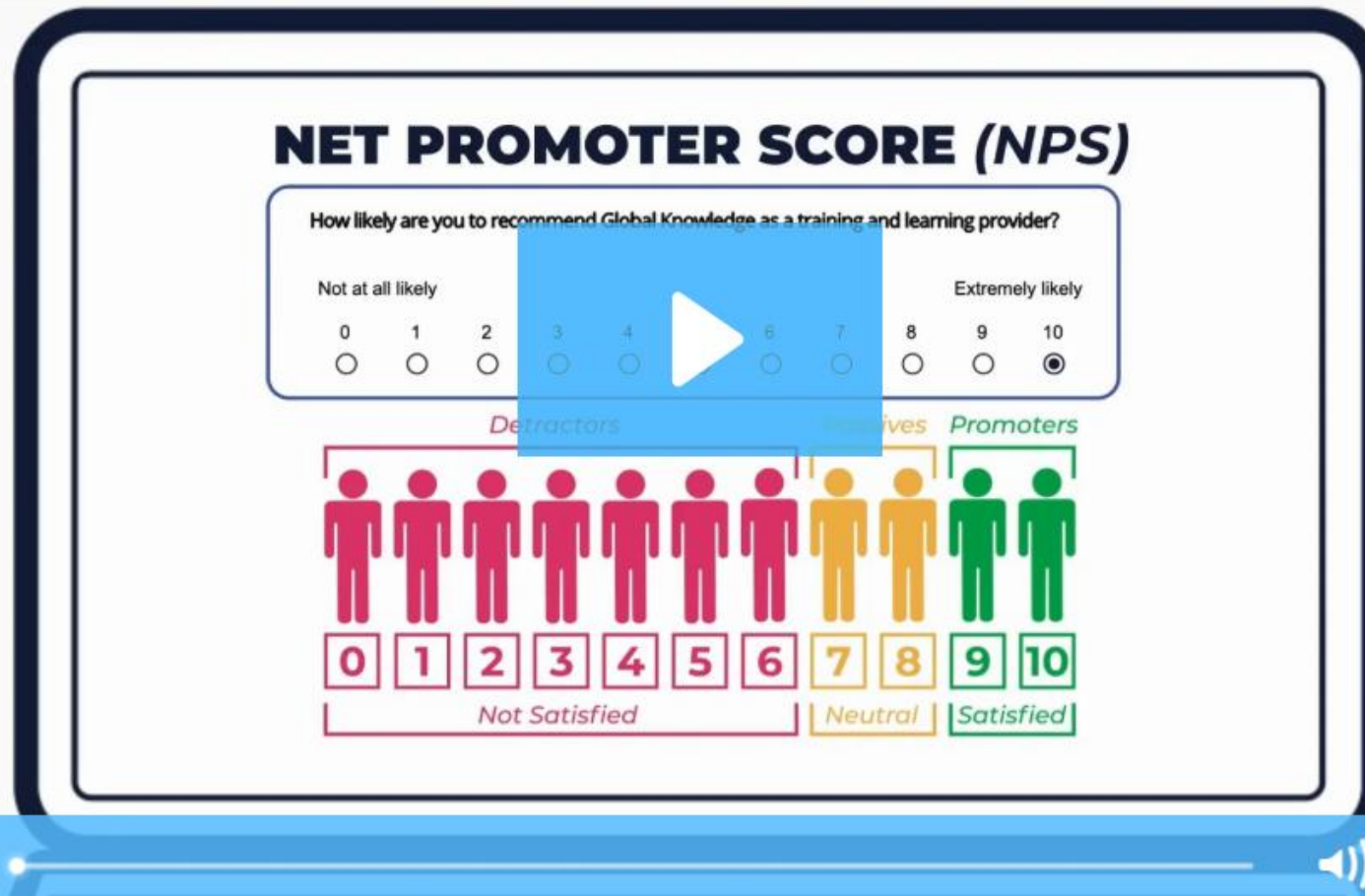


<https://www.linkedin.com/in/mikelively-quantumai/>

About Me: Mason HS AI+ Design Thinking



Evaluation



<https://globalknowledge.wistia.com/medias/ycdgbh9mgo>

https://surveys.skillsoft.com/jfe/form/SV_2l9xxtKvLOyJCKN?EVENTID=WO0000266397

About You

(1 minute intro)



Name and Job function



Your SQL or Python Experience



Why you're taking this course



Any questions so far?



ICE Breaker

What went wrong, or did it?

What emergent behavior can you observe?



About this course

Course description

This course is designed to build your foundational skills in data engineering on Microsoft Fabric, focusing on the Lakehouse concept. This course will explore the powerful capabilities of Apache Spark for distributed data processing and the essential techniques for efficient data management, versioning, and reliability by working with Delta Lake tables, as well as data ingestion and orchestration using Dataflows Gen2 and Data Factory pipelines.

Audience profile

The primary audience for this course is data professionals who are familiar with data modeling, extraction, and analytics. It is designed for professionals who are interested in gaining knowledge about Lakehouse architecture, the Microsoft Fabric platform, and how to enable end-to-end analytics using these technologies.

Flow of the Course

Course Content	Timing	Microsoft Learn Content
Course introduction	15	
Explore end-to-end analytics with Microsoft Fabric	15	https://aka.ms/fabric-intro
Get started with lakehouses	60 with lab	https://aka.ms/fabric-lakehouse
Use Apache Spark	90 with lab	https://aka.ms/fabric-spark
Work with Delta Lake tables	90 with lab	https://aka.ms/fabric-delta
Ingest data with Dataflows Gen2	60 with lab	https://aka.ms/fabric-dataflow
Orchestrate processes and data movement	60 with lab	https://aka.ms/fabric-pipeline
Course conclusion	30	

Lakehouse with Microsoft Fabric



Mike's Companion Site

<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>

Get the most out of your Microsoft Learn profile

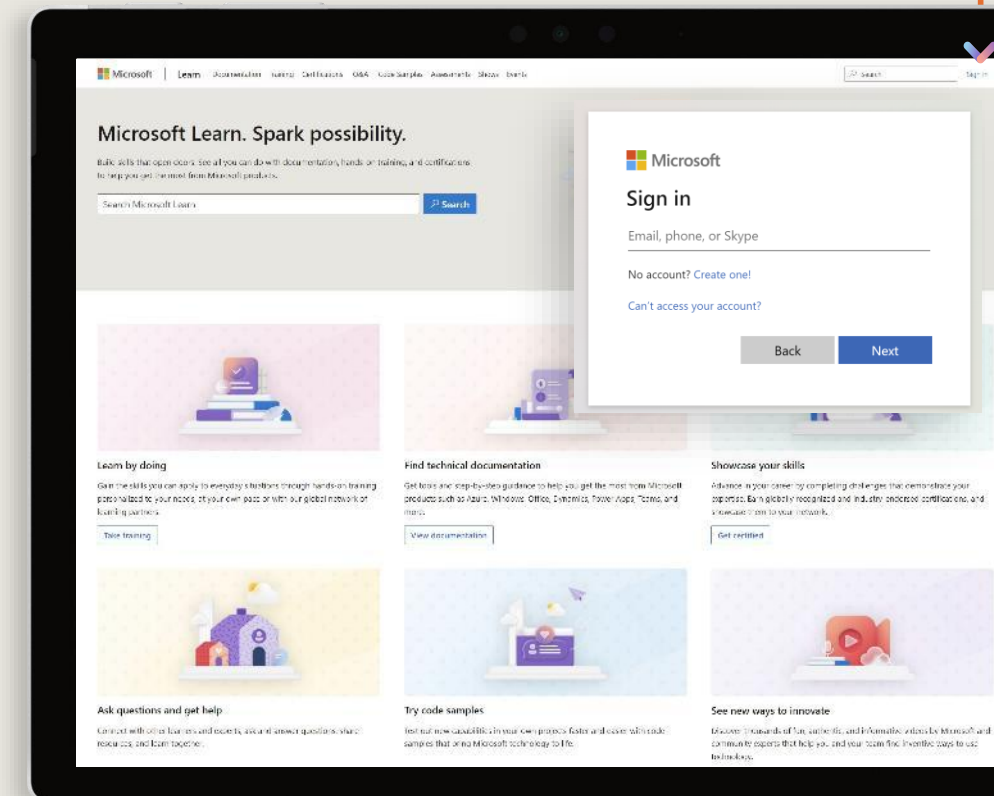
Verify, track, and share your training and certification progress and accomplishments—*all on one platform*.

- Claim your achievement code for this course and share you have completed it.
- Access your course material and track progress on your learning activities.
- Share and verify your Microsoft Credentials via email, on social networking platforms, and on your resume.
- Download and print transcripts and certificates.
- Manage your upcoming activities and credential appointments.

www.aka.ms/MyMicrosoftLearnProfile

Create your Microsoft Learn profile
at learn.microsoft.com

- Select *Sign in* at the top, right corner of any Microsoft Learn page.
- Follow the Microsoft account authentication process.
- If the account that you have chosen to sign-in with doesn't already have a Microsoft Learn profile, you'll be guided to create one.



Access your course material

All course content is available on Microsoft Learn.

learn.microsoft.com/training/courses/browse

- We'll go through this content together and, as the course progresses, I will advise you on which modules to review.
- You can provide feedback for modules on Microsoft Learn. Find out how at the bottom of each page.



Need help? See our [troubleshooting guide](#) or provide feedback for [Microsoft Learn content](#).

Content Devs:

Add lab instructions only. No other edits needed.

Delete this note before publishing.

This course includes labs:

Detailed lab instructions are included in your lab environment.

aka.ms/mslearn-fabric

Part 1: Explore end-to-end analytics with Microsoft Fabric

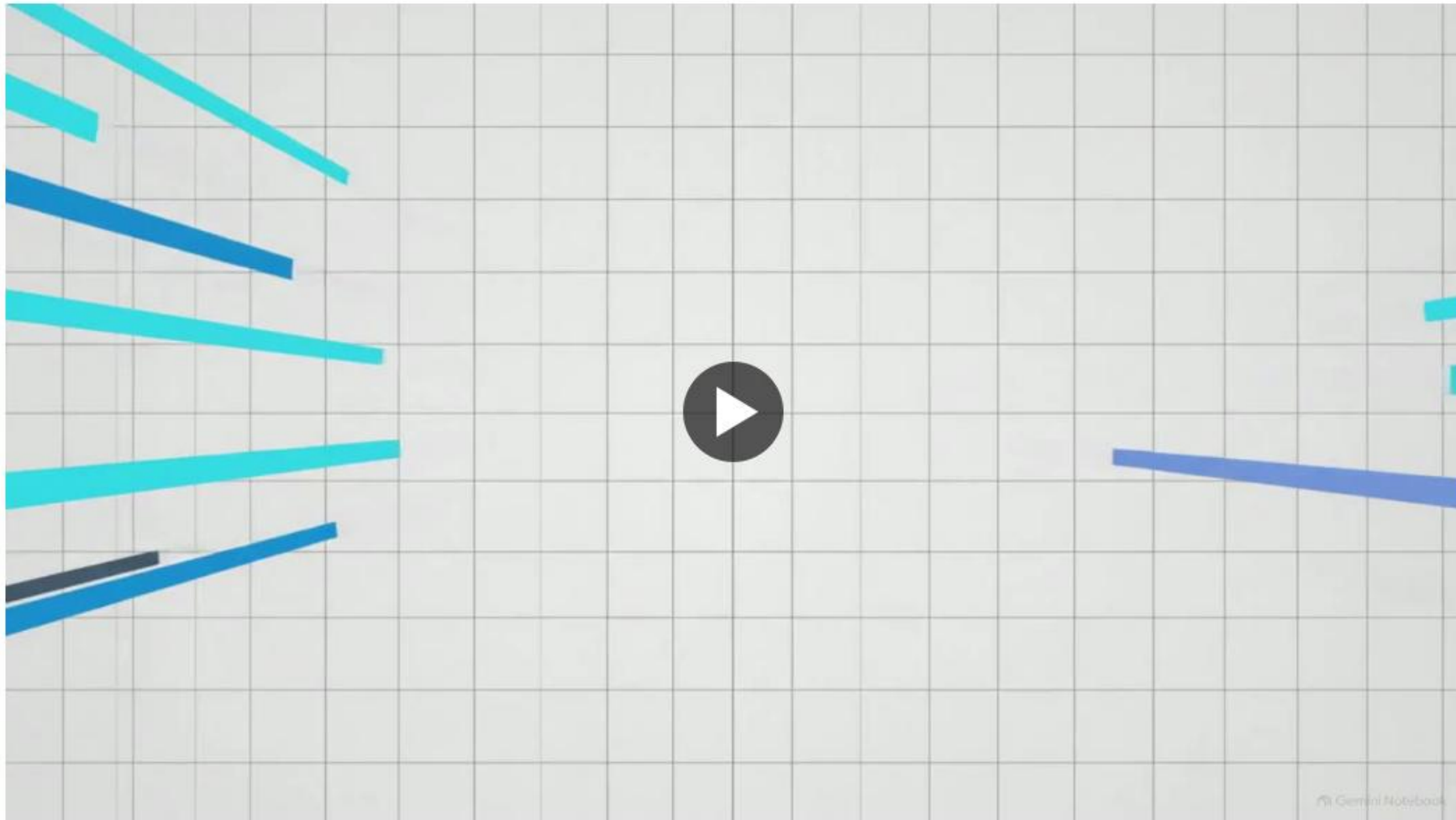


Learning objectives



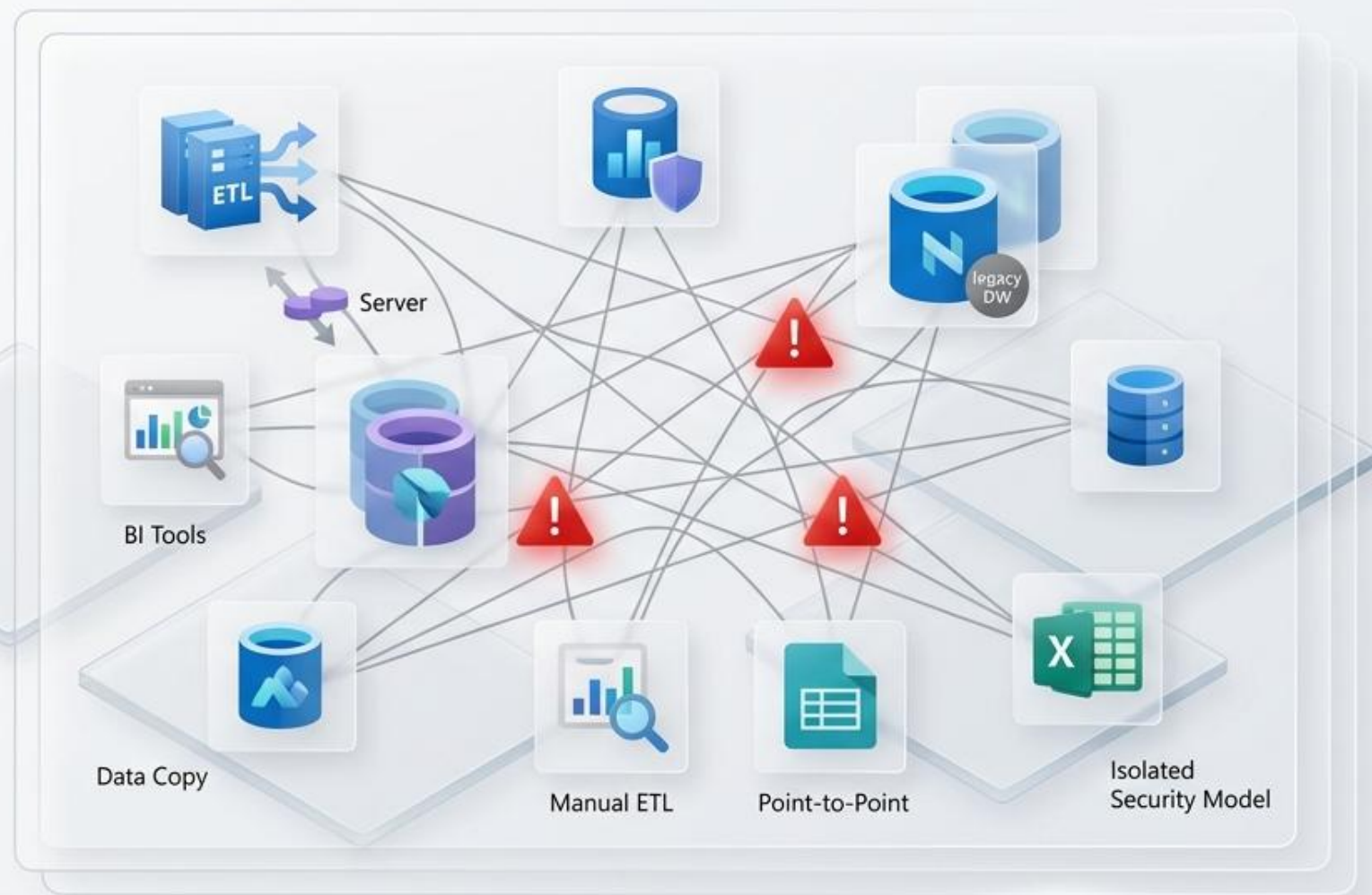
- Describe core features and capabilities of lakehouses in Microsoft Fabric
- Create a lakehouse
- Ingest data into files and tables in the lakehouse
- Query lakehouse tables with SQL

Part 1: End-to-End Analytics with Microsoft Fabric



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>

The Analytics Dilemma: Fragmented Silos Block AI



● Complexity & Cost

- ✓ Maintaining disparate systems and duplicating data across multiple proprietary engines drains engineering resources.

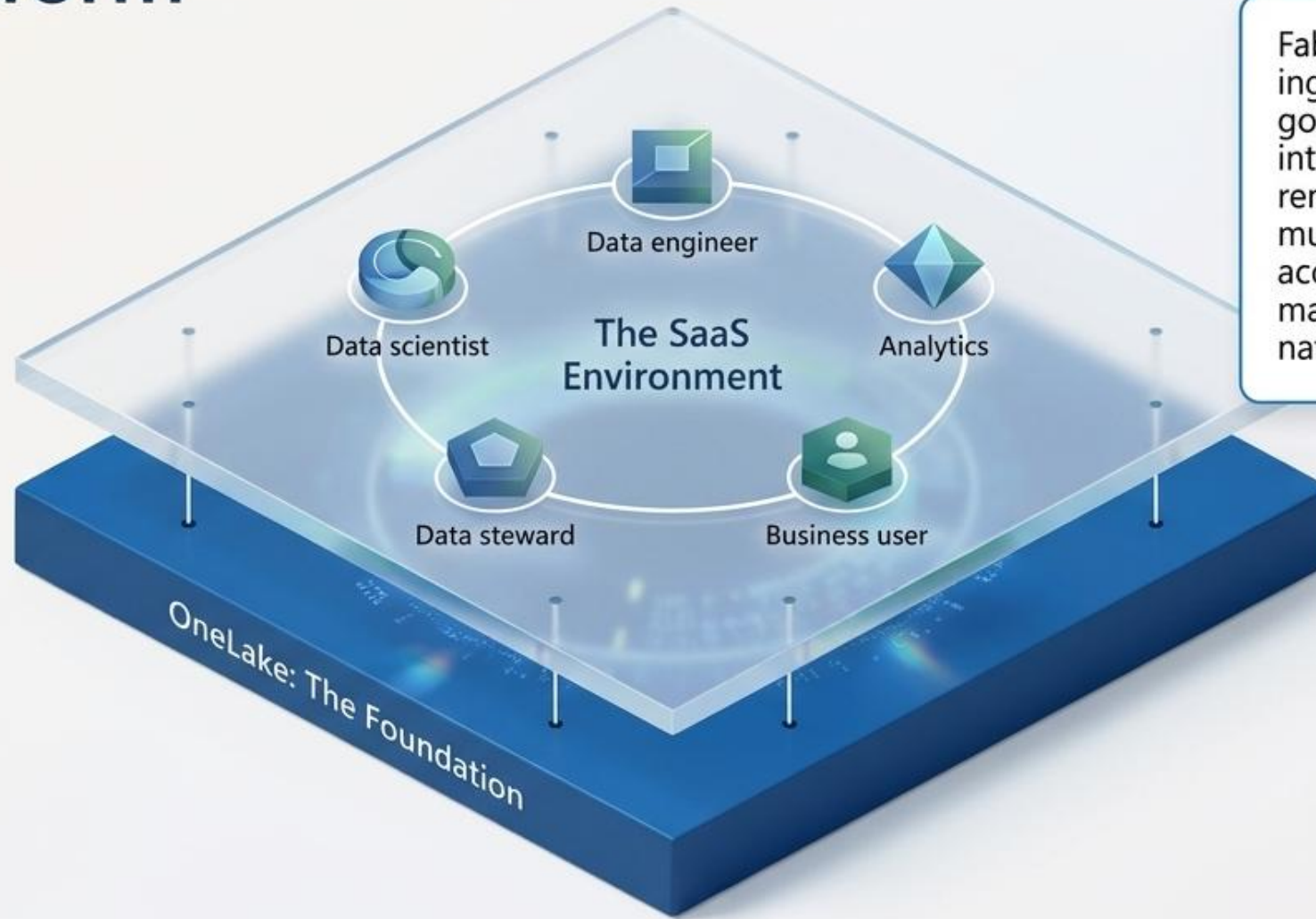
● Governance Gaps

- ✓ Separated security models across platforms create compliance blind spots and disjointed access control.

● AI Roadblock

- ✓ Machine learning models, Copilots, and intelligent agents cannot reason effectively over disjointed, poorly governed data ecosystems.

Microsoft Fabric: The Unified Analytics SaaS Platform



Fabric integrates data ingestion, preparation, governance, and analysis into one environment. By removing data silos and multiple systems, it accelerates workflows and makes enterprise data natively ready for AI.

OneLake & Open Formats: One Copy, Any Engine



All compute engines in Fabric automatically store and interact with data in a single open format: Delta-Parquet. Because tabular data is written in this universal format, all analytical engines interact with it seamlessly. Zero data duplication. Zero vendor lock-in.

Empowering the Modern Data Team

Persona	Traditional Challenge	The Fabric Workflow	Direct AI Contribution
 Data Engineer	Siloed ETL tools	 Uses Pipelines & Notebooks directly on OneLake	 Builds the clean, well-governed foundational data layer.
 Analytics Engineer	Disconnected semantic layers	 Curates data assets in Lakehouses	 Provides the business context for AI accuracy.
 Data Analyst	Downstream ETL bottlenecks	 Uses Direct Lake mode & Power BI	 Translates data into interactive business signals.
 Data Scientist	Integrating ML with rigid systems	 Native Azure ML integration on Lakehouses	 Generates predictions used as grounding data for Agents.

The Collaborative Workflow: One Platform, Zero Bottlenecks

The Data Relay



1. Ingest

Runs a pipeline depositing raw data into OneLake.



2. Prepare

Instantly accesses the same Delta-Parquet file to build a Semantic Model.



3. Analyze

Uses Direct Lake mode in Power BI to query the exact same model with zero downstream transformation.



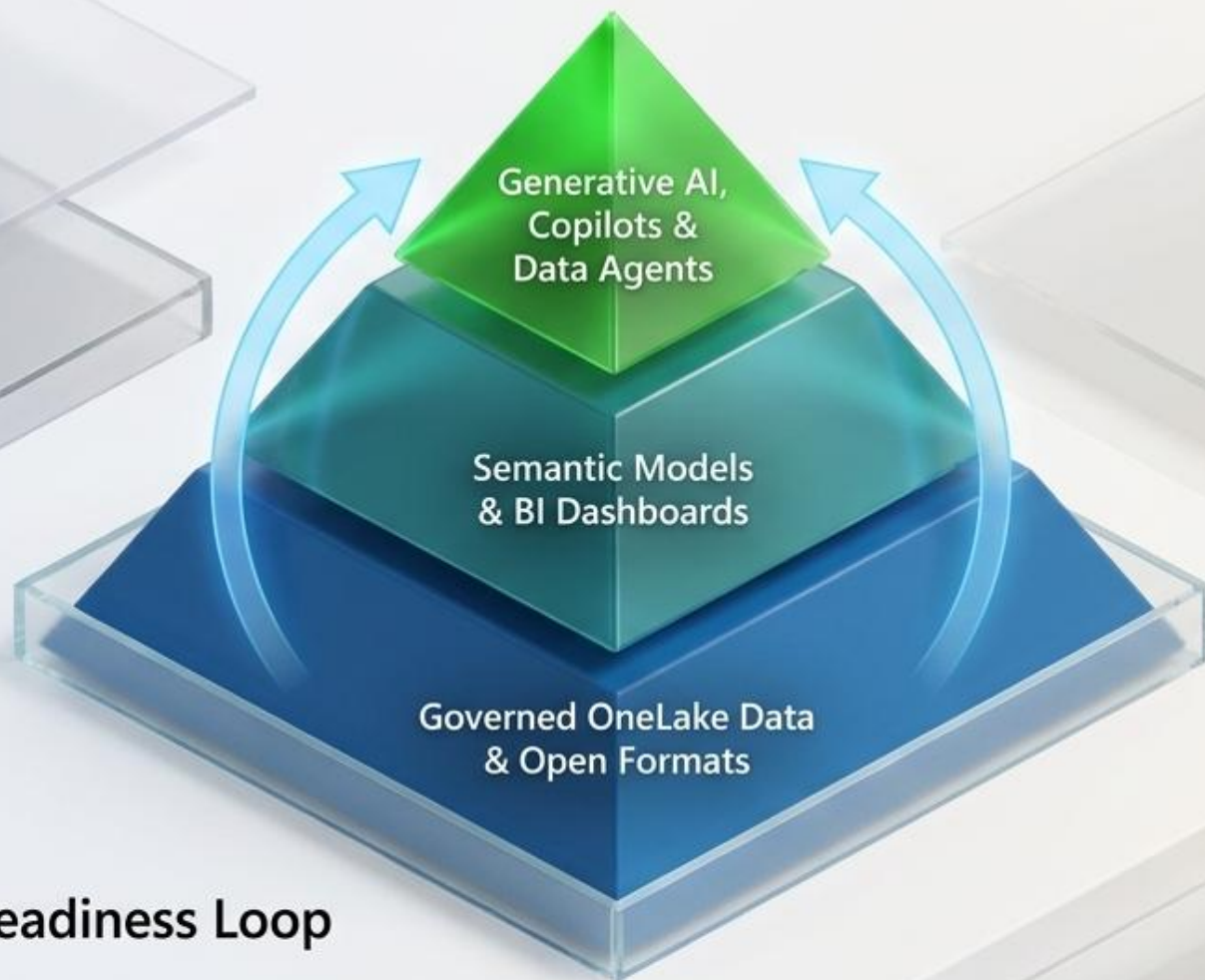
4. Predict

Attaches a Python/Spark Notebook to the same Lakehouse to train a machine learning model.

Instead of passing data copies over a wall, the team collaborates around a single unified dataset.

The AI Synthesis: Analytics Prep Is AI Prep

Analytics Prep Is AI Prep



AI Readiness Loop

Because all data is stored in OneLake using an open format, AI capabilities can access the same governed data as your reports without requiring separate, parallel data preparation pipelines.

The work your data engineers and analysts do to govern and structure data for BI is the exact same work that makes your organization AI-ready. BI Prep = AI Prep.

Copilot in Fabric: Generative AI for Data Professionals



Code Completion & Generation

Copilot provides intelligent Python/Spark code suggestions in Notebooks, generates SQL from natural language, and translates questions into KQL for real-time analysis.



Data Transformation Guidance

Supports citizen and professional developers by generating code for data wrangling in Data Factory and providing plain-language explanations of complex data logic.

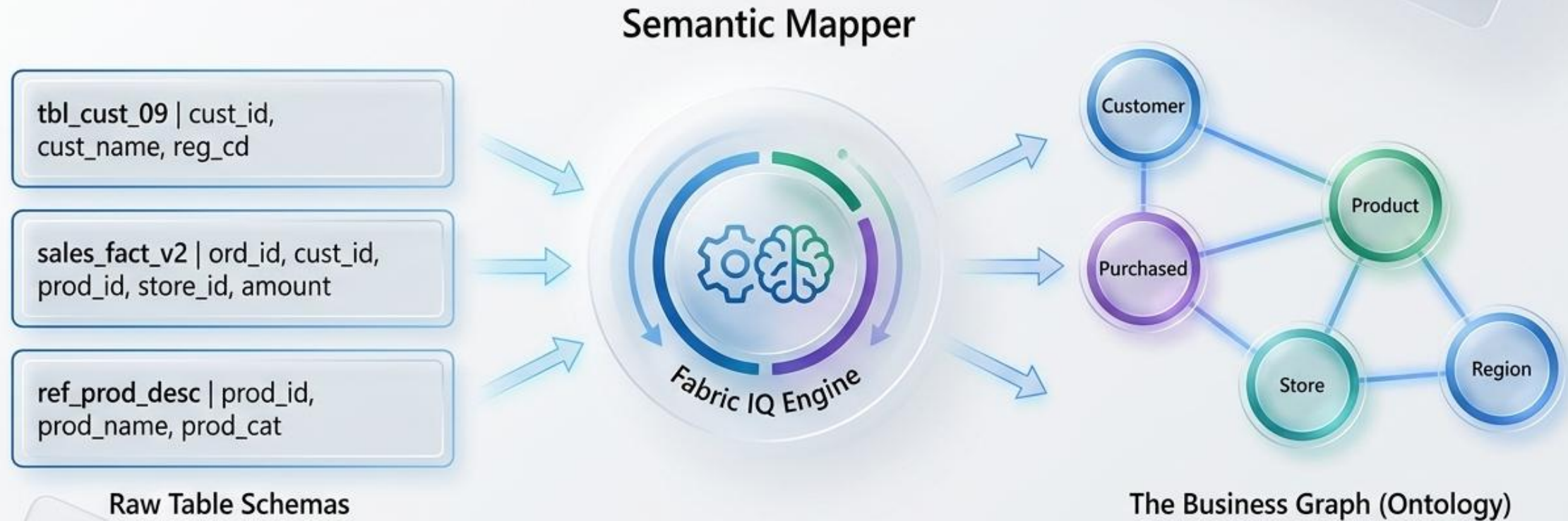


Report & Insight Generation

Automatically generates full Power BI report pages, creates narrative summaries, and enables users to interrogate data via natural language.

Prompt: Given an example of an ontology

Fabric IQ (Preview): Translating Schemas into Business Logic



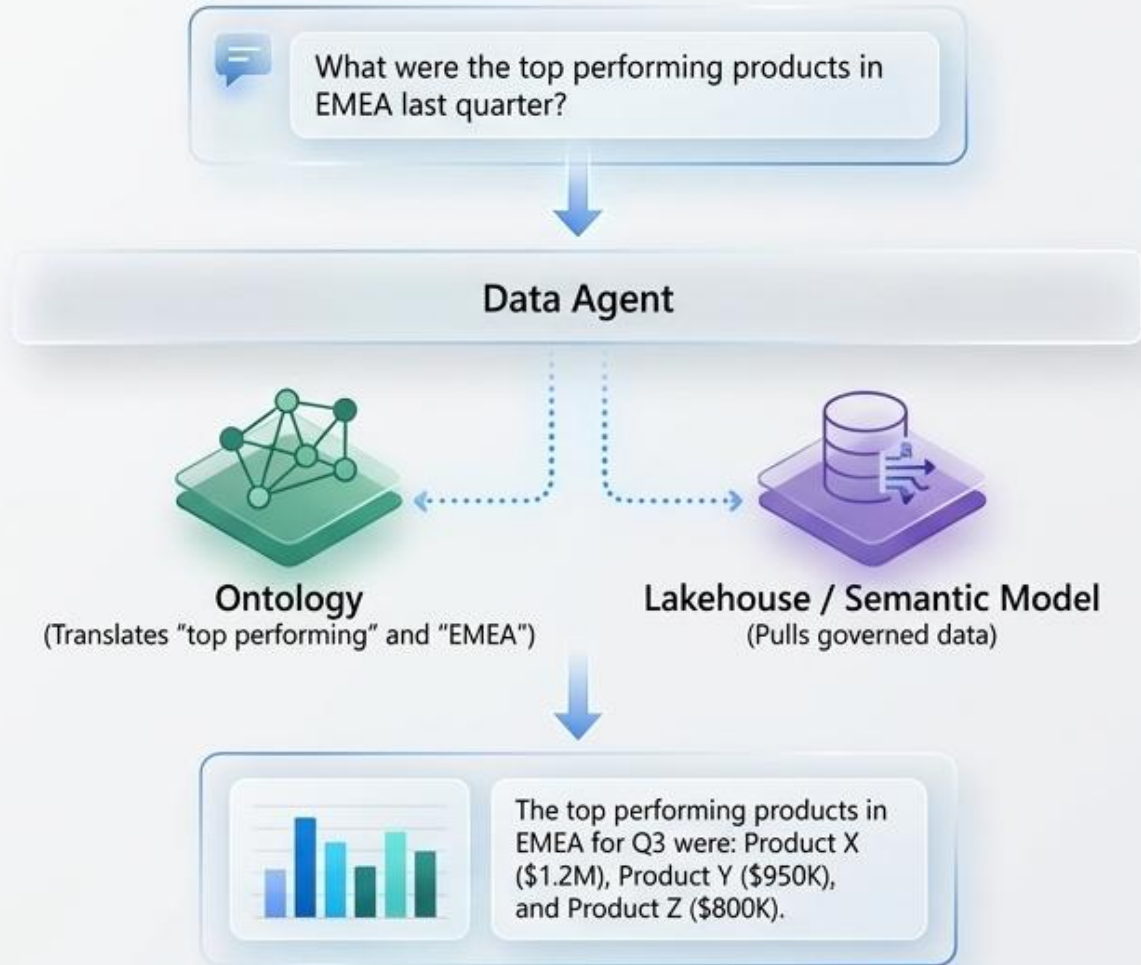
Fabric IQ organizes data according to the language of your business. Its core item is the **Ontology**, which **defines business** concepts, relationships, and rules so AI agents can reason across domains using consistent business language rather than raw table schemas.

The Complete IQ Ecosystem: Context for Enterprise Agents

Workload	Data Source	Core Output	AI Purpose
Fabric IQ	OneLake & Power BI	Ontologies & Semantic Models	Enables Agents to reason over quantitative business data and analytics.
Foundry IQ	Azure, SharePoint, Web	Permission-aware Knowledge	Connects structured & unstructured data for comprehensive enterprise knowledge.
Work IQ	Docs, Meetings, Chats	Collaboration Signals	Gives Agents insight into human workflows and how the organization operates.

Each IQ workload is standalone, but together they provide total organizational context for AI agents.

Fabric Data Agents: Conversational Analytics at Scale



Fabric Data Agents let you build conversational interfaces where users ask questions in natural language.

By connecting directly to your ontology and OneLake, agents provide accurate, governed answers grounded in your unified business data.

Enable and use Microsoft Fabric

Fabric must be enabled in your tenant by either:

- Fabric admin (formerly known as Power BI admin)
- Power Platform admin
- Microsoft 365 admin

Workspaces must then be assigned to Premium per capacity or Fabric license mode.

Part 2: Get started with lakehouses in Microsoft Fabric

<https://learn.microsoft.com/en-us/training/modules/get-started-lakehouses/>



Learning objectives



- Describe end-to-end analytics in Microsoft Fabric
- Understand data teams and roles that use Fabric
- Describe how to enable and use Fabric

Part 2: Microsoft Fabric Lakehouses Guide



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>



Data Lake

- Flexible and highly scalable
- Handles unstructured/semi-structured data natively
- Drawback: Lacks rigid structure for performance-driven analytics



Data Warehouse

- High-performance, strong analytical capabilities
- Highly structured transactional data
- Drawback: Struggles with diverse data formats; expensive to scale



The diagram illustrates the Fabric Lakehouse architecture. At the top, two interlocking gears represent the processing engines. Below them, a central diamond-shaped platform sits on a wide base. Two glowing, wavy lines with arrows indicate data flow from the engines down to the central platform and then outwards to the sides. The entire structure is set against a dark blue background with a grid pattern and glowing circuit lines.

**Apache Spark &
SQL Engines**

**The Fabric
Lakehouse**

Brings database-like
capabilities directly to
your data lake.

OneLake Storage Foundation

Takeaway: Eliminates the need to maintain separate systems for different workloads while supporting both structured and unstructured data.

Tables (The House)

- Stores structured Delta Lake tables.
- Supports direct SQL queries via analytics endpoint.
- Enforces schema and ACID transactions.
- Directly accessible by Power BI.

Fabric Explorer

Type Inert

- > Lakehouse
- ✓ DP601_Bronze
- Folder Tables
- > Folder Files
- > Lakehouse

Files (The Lake)

- Stores raw/semi-structured files (CSV, JSON, Parquet).
- No schema enforcement.
- Ideal for staging and flexible pre-transformation data exploration.

The Engine: Delta Lake Capabilities



ACID Transactions

Ensures data consistency and reliability even when multiple users read and write data simultaneously.



Delta Lake is an open-source storage layer that organizes files into structured queryable formats.

Schema Enforcement

Validates incoming data against the defined table schema, automatically preventing corrupt data ingestion.



Time Travel

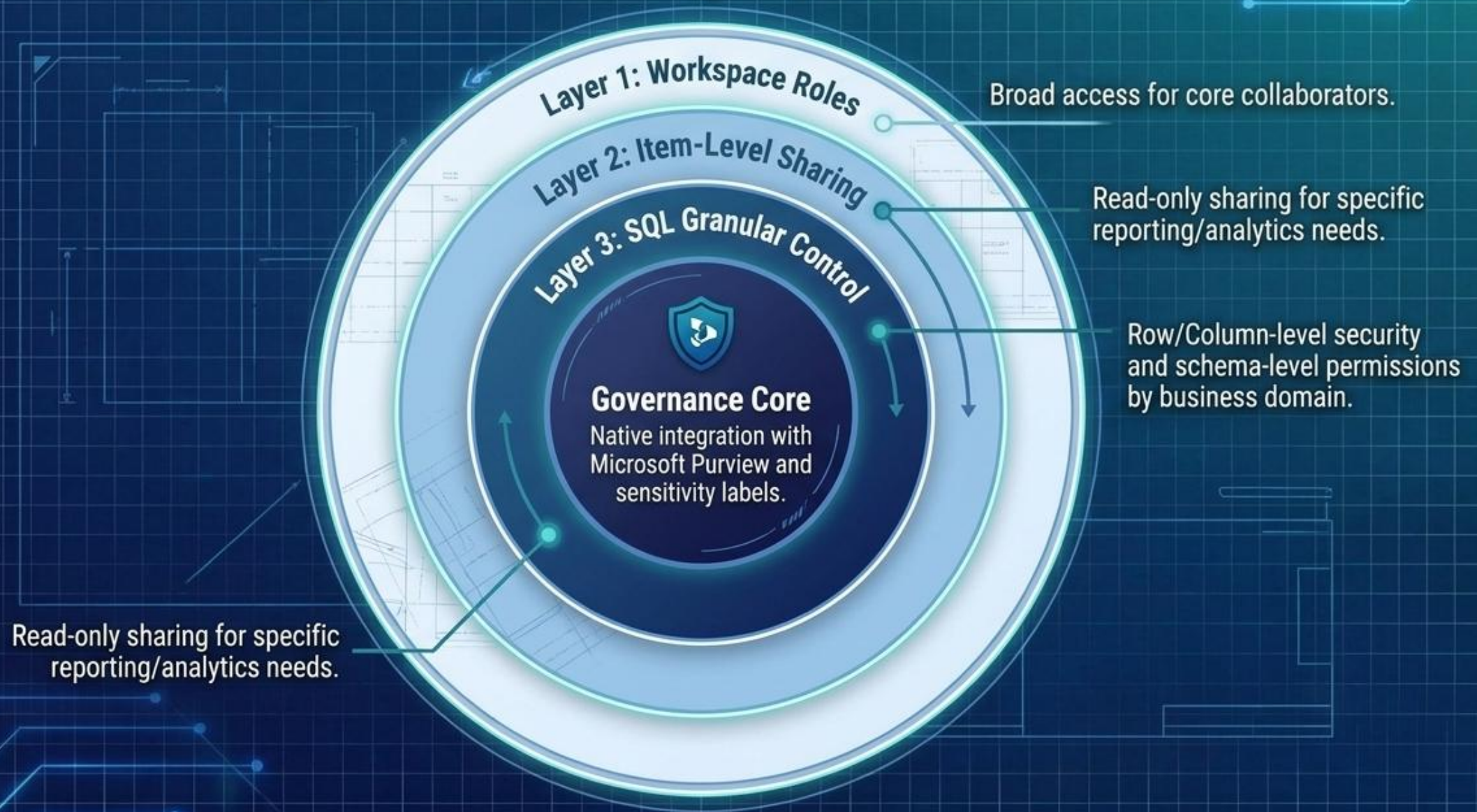
Maintains a robust transaction log, allowing users to query previous versions of data or seamlessly roll back changes.



Efficient Operations

Unlike traditional flat files, Delta tables natively support high-speed update and delete operations directly on the lake.

Layered Security & Governance



Lifecycle Step 1: Ingesting Data

**Business Users
(No-Code)**

Direct Upload via Lakehouse Explorer

Use 'Load to tables' to auto-convert CSV/Parquet to Delta tables instantly.



**Analysts / BI
(Low-Code)**

Dataflows Gen2 & Data Pipelines

Use familiar Power Query interfaces or visual copy data activities.



**Data Engineers
(Pro-Code)**

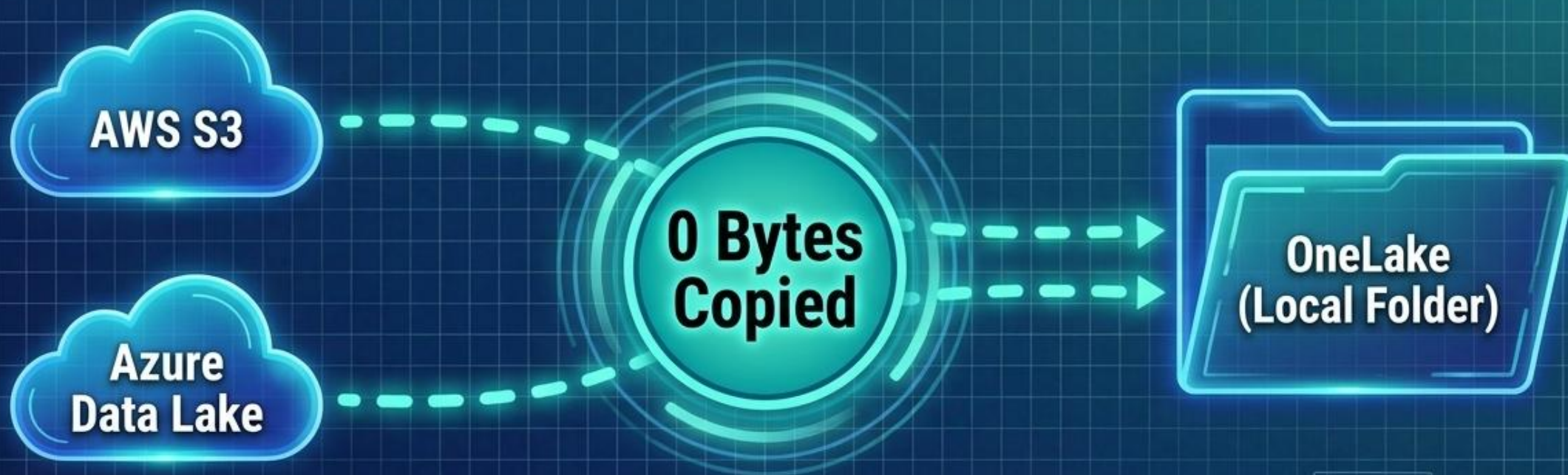
Apache Spark Notebooks

Programmatic, complex ETL ingestion at scale.



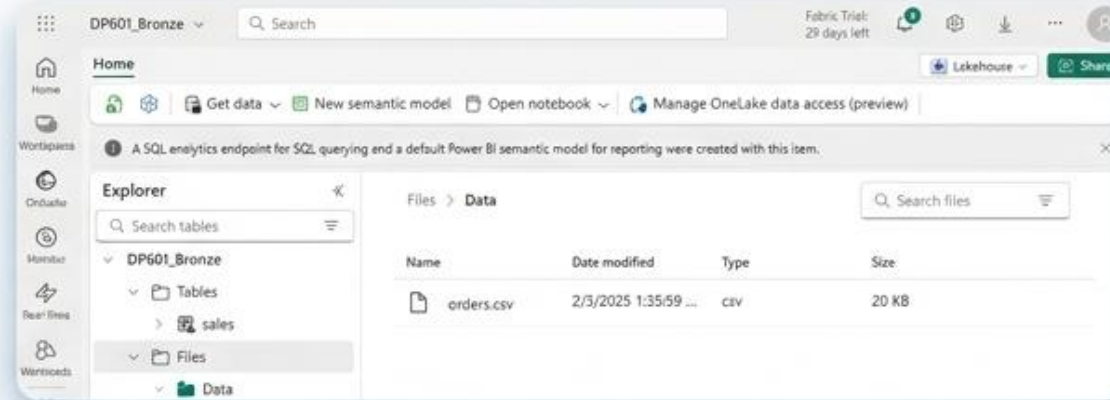
The Lakehouse

The Power of Shortcuts: Zero Data Movement



What it is	The Benefit	Schema Shortcuts
Virtual references to data in external storage clouds or other Fabric items.	Eliminates data duplication. OneLake acts as the single identity manager to authorize access.	Instantly map an entire external schema directly to a folder of local Delta tables.

Lifecycle Step 2: Transforming Data



Notebooks

For Data Engineers

Programmatic transformation using PySpark, SQL, and Scala. Features built-in Copilot for natural language code generation.

Dataflows Gen2

For BI/Excel Users

Visual, step-by-step transformation utilizing the familiar Power Query interface.

Data Pipelines

For Orchestration

Visual interface to string together multiple ETL activities to run in parallel or sequence.

Lifecycle Step 3: Choose Your Query Engine

	SQL Analytics Endpoint	Lakehouse Query Explorer	Spark Notebooks
Access	Read-only T-SQL	Read & Modify Spark SQL	Read & Modify PySpark / SQL
Best for	Ad-hoc BI queries, data validation, connecting external BI tools, and creating SQL views.	Quick data inspections, testing cross-schema joins, and generating inline charts directly in the UI.	Complex programmatic logic, statistical methods, Machine Learning prep, and saving reusable code.

Engineering Deep Dive: Spark Notebooks

Spark SQL Syntax

```
SELECT * FROM schema.table
```

Use Case: Best for familiar SQL patterns, simple aggregations, and standard data retrieval.

PySpark Syntax

```
df.select().filter()
```

Use Case: Best for complex DataFrame manipulations, applying statistical methods, and deep Python library integration.

Cross-Workspace Power

Query seamlessly across multiple lakehouses in a single query using the 4-part namespace: `workspace.lakehouse.schema.table`

Lifecycle Step 4: Visualize with Power BI Direct Lake



Direct Lake Mode

Power BI reads the underlying Delta Parquet files directly. No data import. No data duplication. Blistering fast query performance.

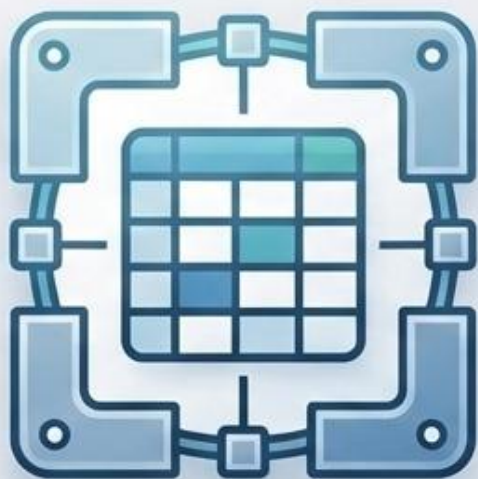
Semantic Models

Define relationships, measures, and business logic directly on top of Lakehouse tables to standardize reporting.

Ad-Hoc Connections

Analysts can still connect legacy BI tools or Excel directly to the SQL endpoint.

The Synthesis: Good Engineering Fuels AI



Well-Structured Lakehouse
(Tables & Schemas)

Fabric IQ Data Agents

AI translates natural language into accurate SQL by relying entirely on clean, consistent table schemas.

Copilot in Power BI

Auto-generates reports and answers complex questions by reasoning over well-defined semantic models.

M365 Integration

Lakehouse models directly feed natural language data exploration across Microsoft 365 experiences.

Insight: The investment in data organization pays dividends beyond immediate analytics—it is the foundation for intelligent experiences.

Work with a Fabric lakehouse

The screenshot displays the Microsoft Fabric user interface. At the top, a table header lists various columns: Name, Type, Task, Owner, Refreshed, Next refresh, Endorsement, Sensitivity, and Included in app. Below this, a navigation bar includes icons for Home, File, Refresh, Explore this data, Analyze in Excel, Lineage, Open data model, and Write DAX queries. The main workspace is divided into several sections:

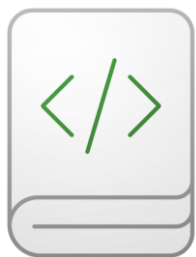
- Home** and **Reporting** tabs are visible at the top of the workspace.
- A **SQL analytics endpoint** dropdown is located in the top right of the workspace.
- The **Explorer** pane on the left shows a tree view of the data lakehouse structure, including Warehouses, Schemas (dbo, INFORMATION_SCHEMA, queryinsights, sys), Security, and Queries (My queries, Shared queries). It also lists Model layouts.
- The **SQL query 1** editor in the center shows the following query:

```
1 SELECT TOP 10 *
2 FROM sales
3 WHERE OrderQty > 20
```
- Below the query editor, the **Results** tab is active, displaying a table of query results.

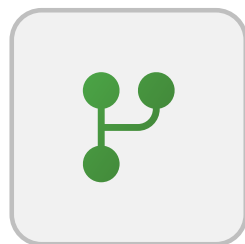
	123 SalesOrderID	OrderDate	123 CustomerID	123 Lineltem	123 ProductID	123 OrderQty	12F LineltemTotal
1	71783	2022-06-02	29957	7	976	25	19136.14
2	71784	2022-06-03	29736	39	864	23	763.11
3	71797	2022-07-01	29796	33	864	23	763.11

Load data into a lakehouse

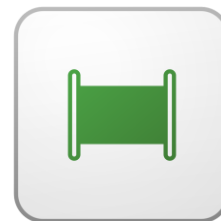
Ingest and connect to data sources to transform and load for analytics.



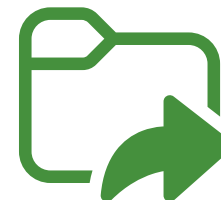
Notebooks



Dataflows Gen2



Pipelines



Shortcuts

Explore, transform, and visualize data in the lakehouse

Tools and techniques to explore and transform data:

- ★ Apache Spark

 - 📄 Notebooks

 - 📅 Spark Job Definitions

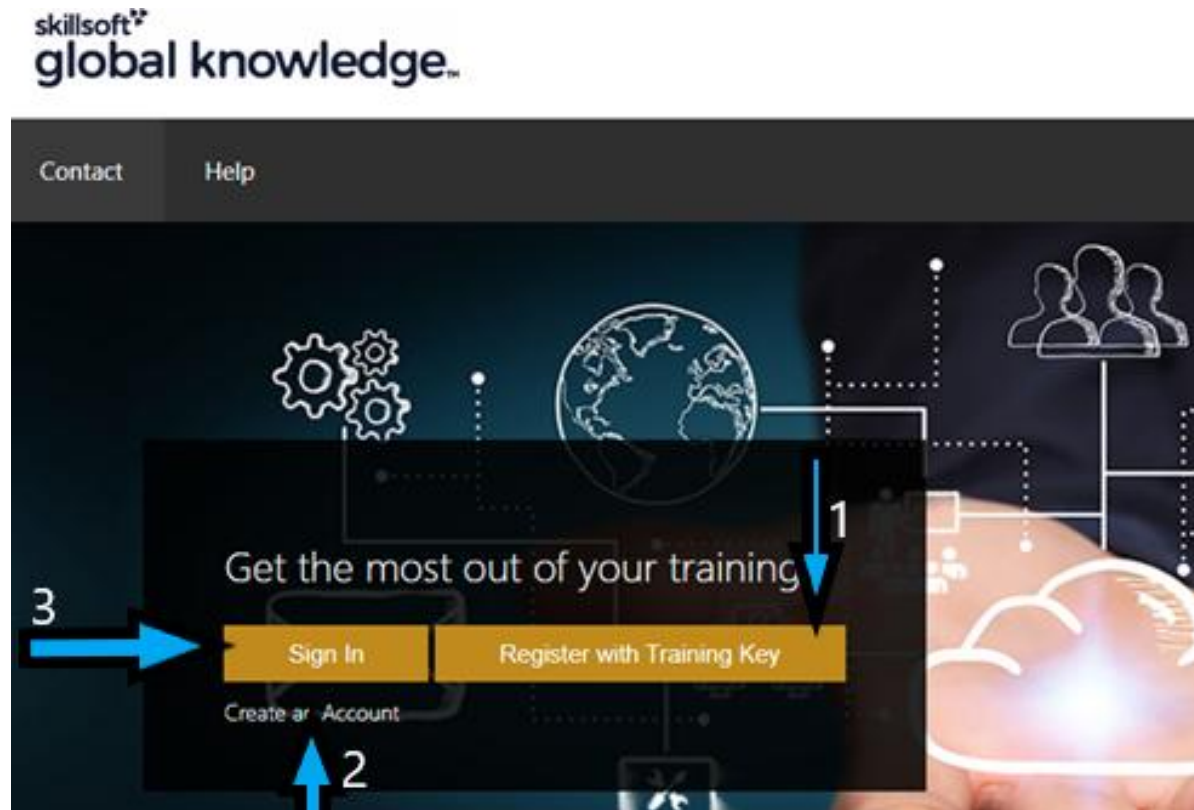
- 🏠 SQL analytics endpoint

- 🔗 Dataflows Gen2

- 📡 Data Pipelines

Visualize lakehouse data using Power BI

Exercise environment



<https://globalknowledge.learnondemand.net/>

Exercise environment

Student Access to Labs

Training Key: *****

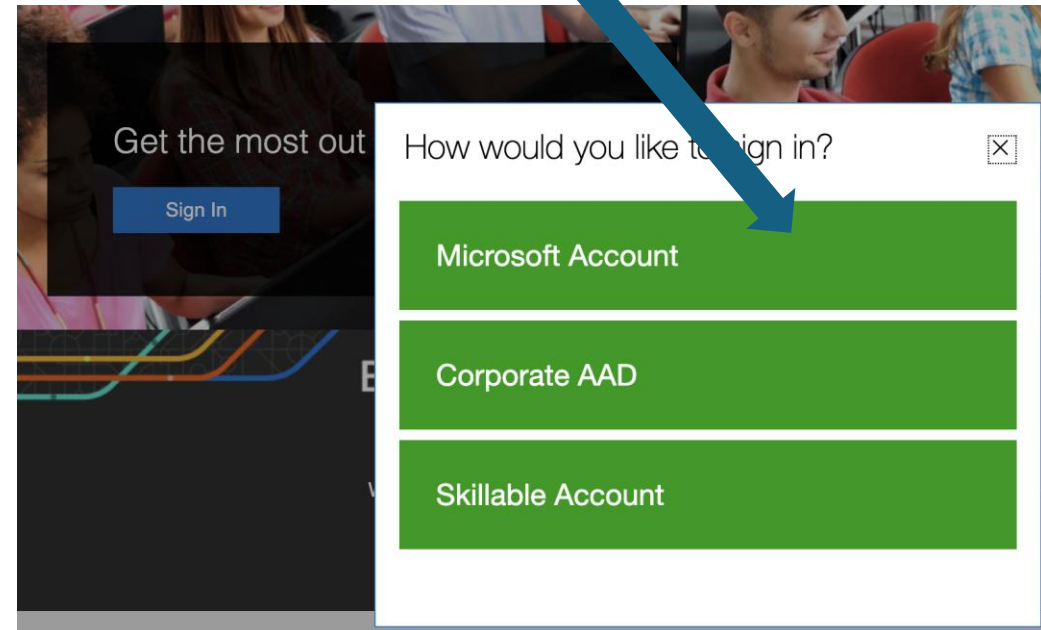
In order to access course materials, ask students to navigate to

<https://globalknowledge.learnondemand.net/>

.From there, students should sign in with a Microsoft Account (Live ID). Students should always login with a Microsoft Account. After signing in, students should click

➡ **Redeem Training Key at the top of the page** and follow the prompts which will give students access to course materials.

Personal Account



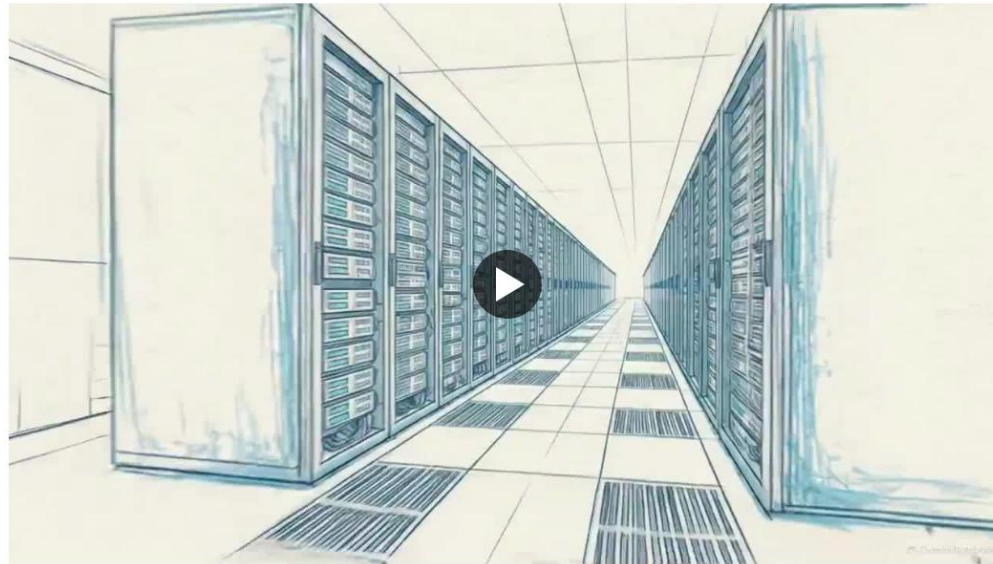
Exercise

30 minutes



Lab 1: Create and ingest data with a Microsoft Fabric lakehouse

Lab 1: Microsoft Fabric Lakehouse Essentials



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-labs-michael-lively-ejlx/>

Part 3: Use Apache Spark in Microsoft Fabric

<https://aka.ms/fabric-spark>



Learning objectives



- Describe end-to-end analytics in Microsoft Fabric
- Understand data teams and roles that use Fabric
- Describe how to enable and use Fabric

Part 3: Apache Spark in Microsoft Fabric

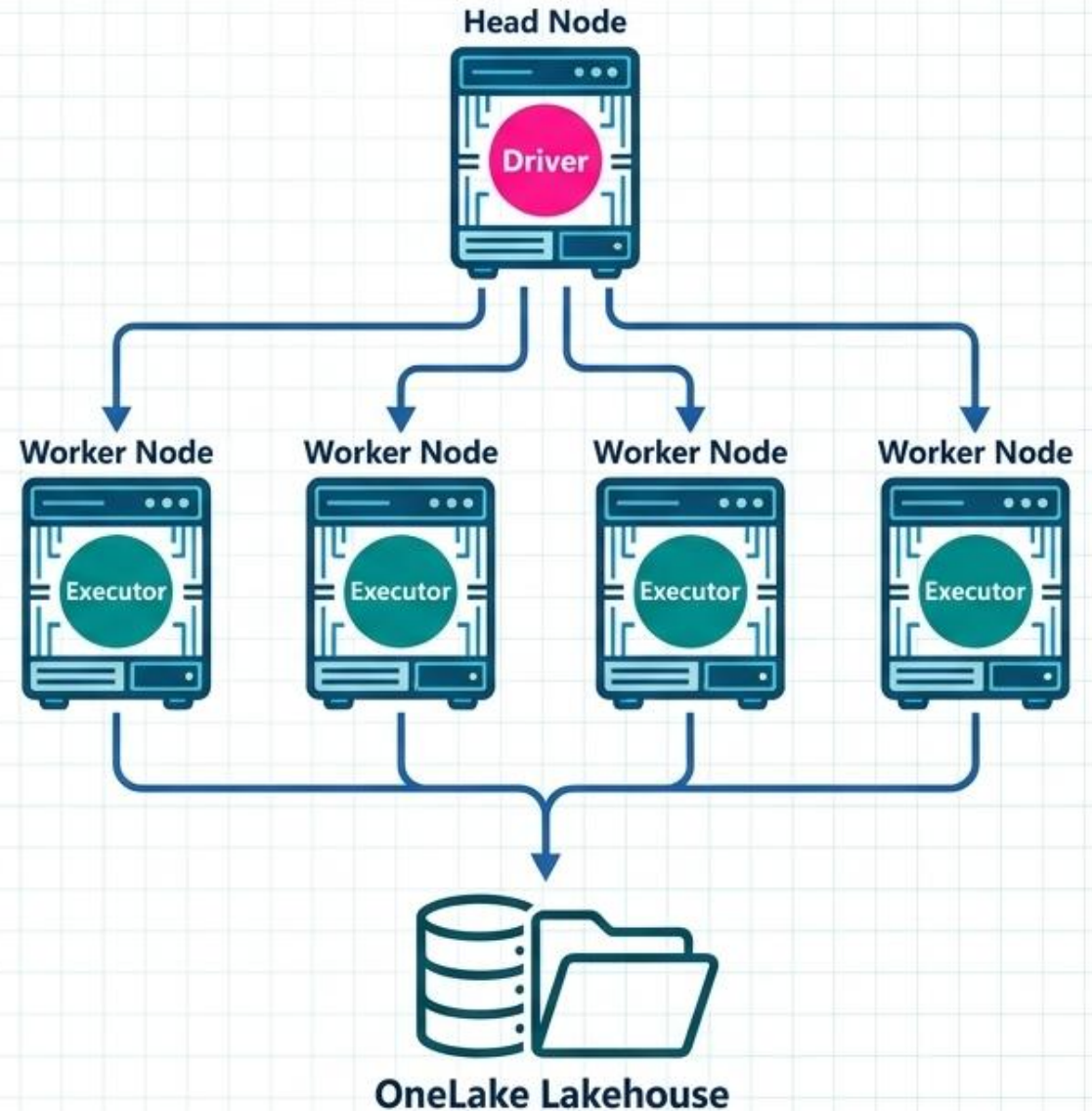


<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>

The Architecture of Distributed Processing

- Apache Spark is a parallel processing framework for large-scale data analytics.
- Bypasses single-machine bottlenecks using a 'divide and conquer' execution model.
- Microsoft Fabric integrates Spark clusters natively into workspaces, enabling direct processing against OneLake data with zero hardware management.

"Divide & Conquer" Architecture Map



Workspace settings

- General
- License info
- Azure connections
- System storage
- Git integration
- OneLake
- Workspace identity
- Network security
- Monitoring

Power BI

- General
- Data connections
- Embed codes

Data Engineering/Science

- Spark settings

Spark settings

Configure and manage settings for Spark workloads and the default environment for the workspace.

Pool Environment Jobs High concurrency Automatic log

Default pool for workspace

Use the automatically created starter pool or create custom pools for workspaces and items in the capacity. If the setting Customize compute configurations for items is turned off, this pool will be used for all environments in this workspace.

StarterPool

Pool details		
Node family	Node size	Number of nodes
Memory optimized	Medium	1 - 10

Customize compute configurations for items ☒ On

When turned on, users can adjust compute configuration for individual items such as notebooks and Spark job definitions. [Learn more about Customize compute configurations for items](#)

Fabric provides a pre-configured Starter Pool for immediate execution, with full support for specialized Custom Pools.

Node Family

Specifies the virtual machine type. Memory-optimized nodes are the default recommendation for optimal performance.

Autoscale

The elasticity rule. Dictates whether the pool automatically provisions nodes on demand between initial and maximum boundaries.

Dynamic Allocation

The resource manager. Determines whether executor processes dynamically scale across worker nodes based on live data volumes.

The Execution Matrix



	Notebooks	Spark Jobs
Interaction Type	Interactive, exploratory	Non-interactive, automated
Primary Use Case	Data exploration, collaborative analysis, visualization	Scheduled ingestion, automated batch transformations
Code Structure	Multi-language cells using Magics	Single uploaded script (.py) + reference files

Both tools utilize the exact same underlying Spark Pools and Environments configured in the Fabric workspace.

Blueprint Loom

Infer Schema

```
df = spark.read.option("header", "true")  
  .csv("abfss://[redacted] header="true")  
  .csv("abfss://.../sales.csv")
```

Relies on CSV header; incurs a slight performance penalty as Spark scans data to guess types.

Explicit Schema

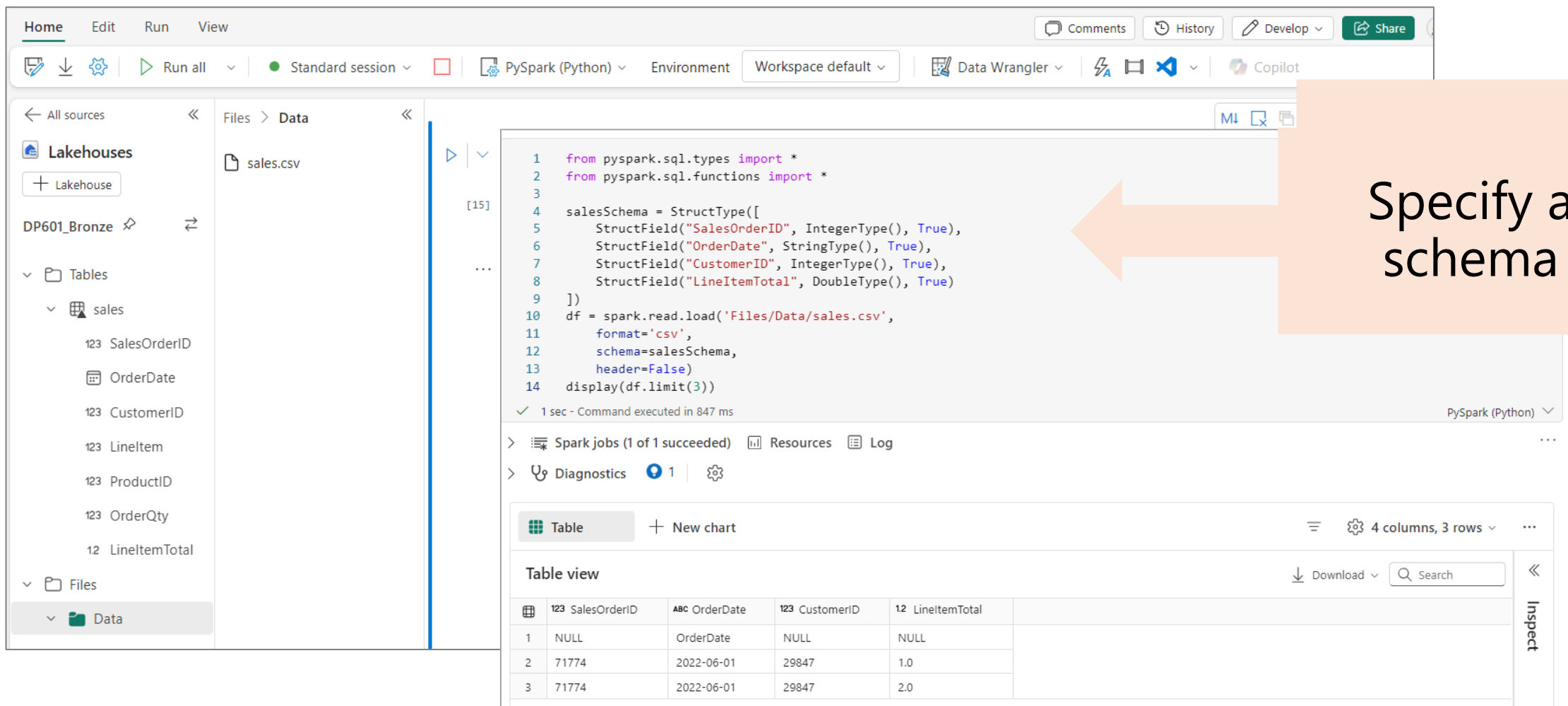
```
from pyspark.sql.types import StructType,  
    StructField, StringType, IntegerType,  
    DateType, DecimalType;  
  
salesSchema = StructType([  
    StructField("SalesOrderID", IntegerType(), True),  
    StructField("OrderDate", DateType(), True),  
    ...  
])
```

Defines exact datatypes (IntegerType, StringType); significantly faster and safer for large datasets.

The Spark Dataframe (part of the Spark SQL library) is the primary structure for tabular data. While Spark can infer data structures, explicitly defining a schema maximizes ingestion performance.

Load data in a Spark Dataframe

Schema can be either inferred or specified



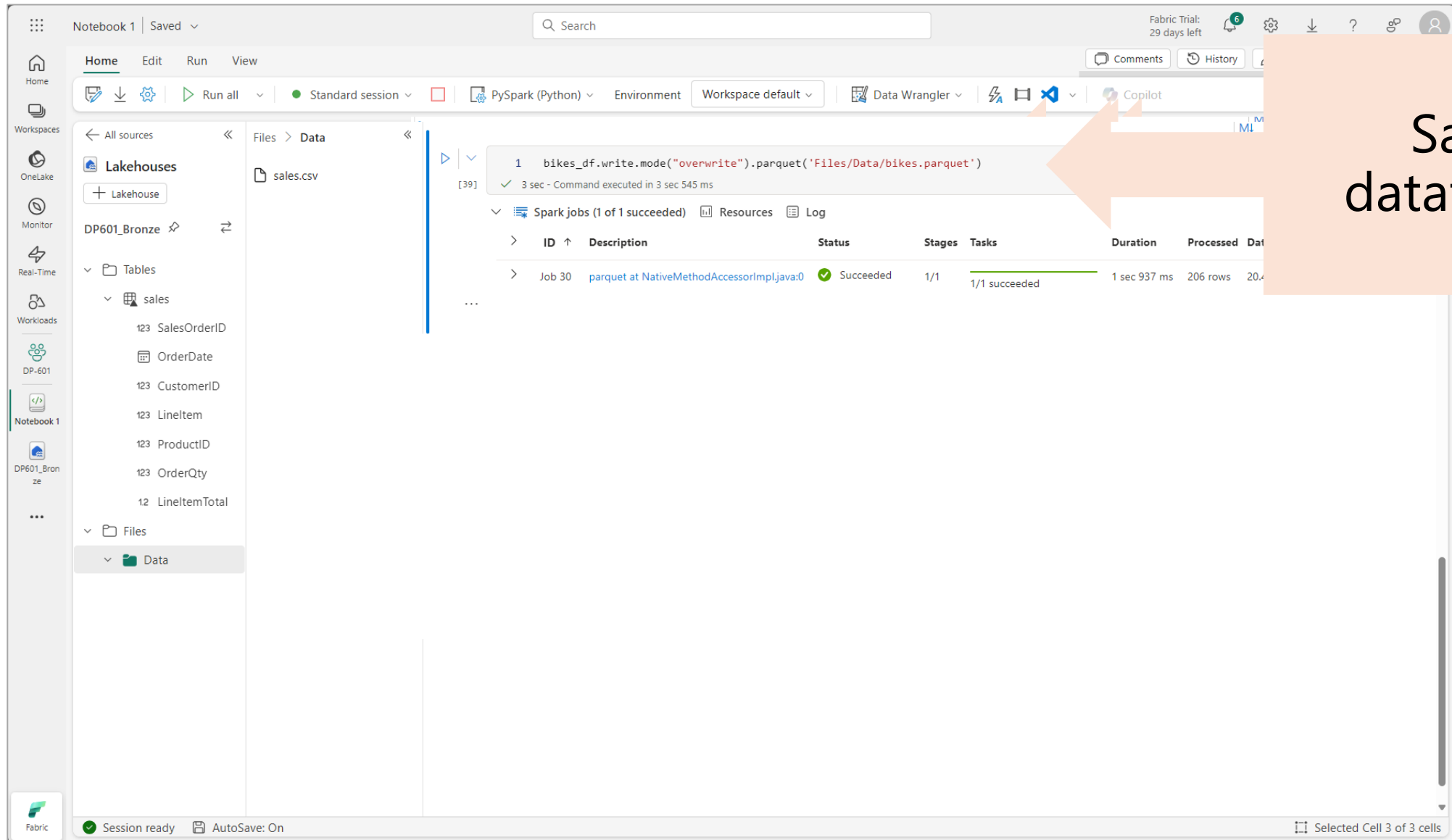
The screenshot shows a Databricks workspace interface. On the left, the 'Files' tab is selected, showing a file named 'sales.csv' under the 'Data' directory. The main editor displays a PySpark script that loads the CSV file into a DataFrame, explicitly defining a schema. An orange callout box with the text 'Specify a schema' points to the schema definition in the code.

```
1 from pyspark.sql.types import *
2 from pyspark.sql.functions import *
3
4 salesSchema = StructType([
5     StructField("SalesOrderID", IntegerType(), True),
6     StructField("OrderDate", StringType(), True),
7     StructField("CustomerID", IntegerType(), True),
8     StructField("LineItemTotal", DoubleType(), True)
9 ])
10 df = spark.read.load('Files/Data/sales.csv',
11     format='csv',
12     schema=salesSchema,
13     header=False)
14 display(df.limit(3))
```

Below the code editor, the 'Table' view is displayed, showing the first three rows of the loaded data:

	123 SalesOrderID	ABC OrderDate	123 CustomerID	12 LineItemTotal
1	NULL	OrderDate	NULL	NULL
2	71774	2022-06-01	29847	1.0
3	71774	2022-06-01	29847	2.0

Transform data in a Spark Dataframe



The screenshot displays a Databricks notebook interface. The left sidebar shows the workspace structure with 'Lakehouses' and 'Tables' sections. The 'Tables' section is expanded, showing a table named 'sales' with columns: SalesOrderID, OrderDate, CustomerID, Lineltem, ProductID, OrderQty, and LineltemTotal. The 'Files' section is also expanded, showing a folder named 'Data'. The main notebook area shows a code cell with the following command:

```
1 bikes_df.write.mode("overwrite").parquet('Files/Data/bikes.parquet')
```

The command has been executed successfully, as indicated by the green checkmark and the message: "3 sec - Command executed in 3 sec 545 ms". Below the code cell, the Spark jobs section shows a table with the following data:

ID	Description	Status	Stages	Tasks	Duration	Processed	Data
Job 30	parquet at NativeMethodAccessorImpl.java:0	✓ Succeeded	1/1	1/1 succeeded	1 sec 937 ms	206 rows	204

An orange arrow points from the text "Save dataframe" to the code cell, highlighting the action performed.

Data Transformation Pipeline

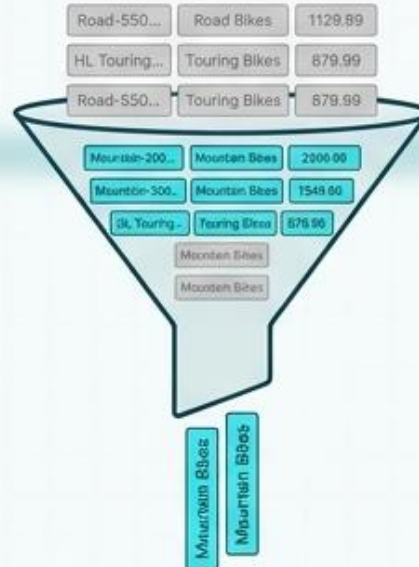
```
.select("ProductName", "Category",  
"ListPrice")
```

Node 1: Select

SeccoOrderID	OrderDate	CustomerID	LineItem	OrderQty	ProductName	Category	ListPrice
					Mountain-200...	Mountain Bikes	2209.99
					Road-550...	Road Bikes	1129.00
					HL Touring...	Touring Bikes	879.99

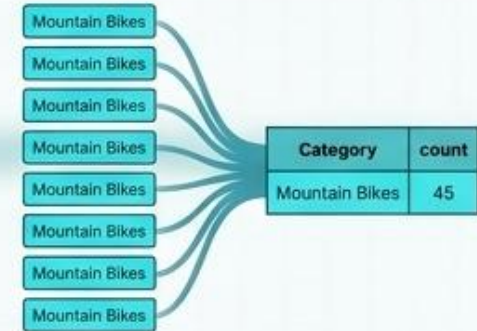
```
.where(df["Category"]=="Mountain Bikes")
```

Node 2: Filter



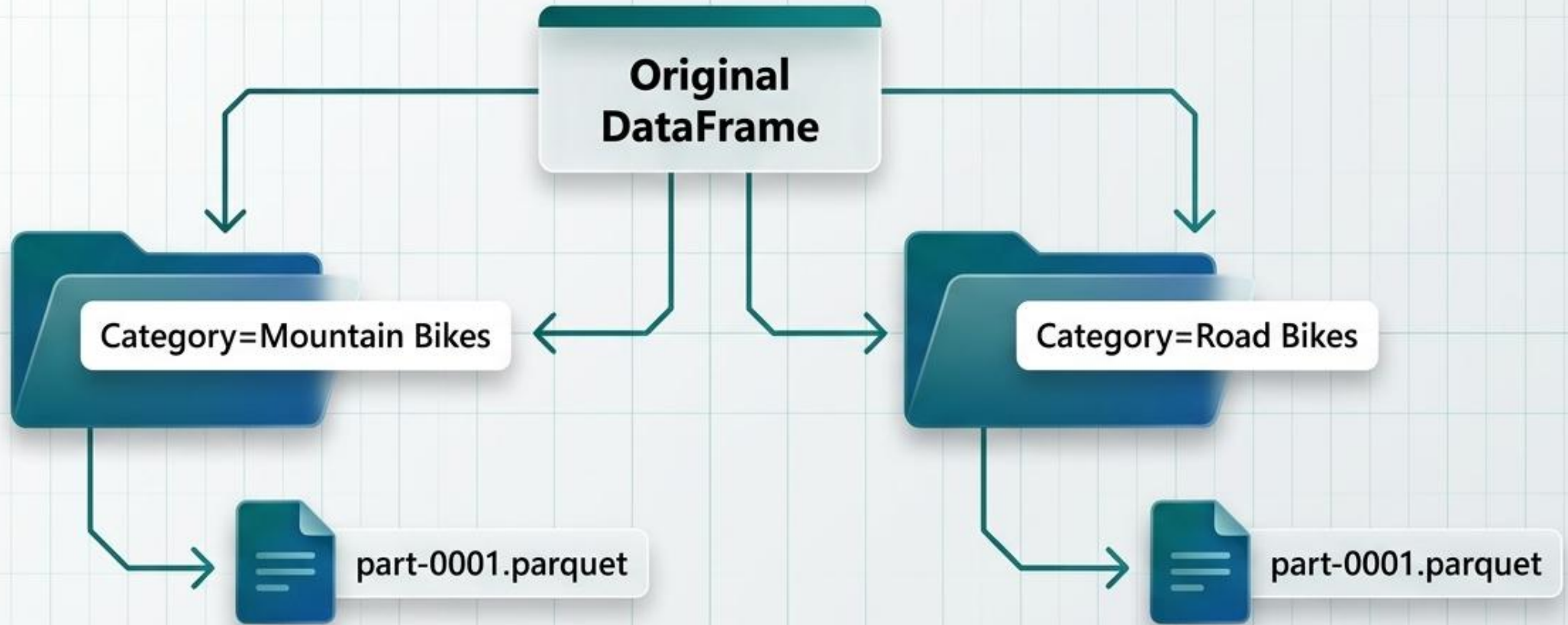
```
.groupBy("Category").count()
```

Node 3: Aggregate



Spark Dataframe methods return new dataframe objects, allowing engineers to cleanly chain operations together in a single, readable execution block.

```
bikes_df.write.partitionBy("Category").mode("overwrite").parquet("Files/bike_data")
```



Partitioning is a critical IO optimization. By structuring output files into distinct physical folders based on column values (e.g., Category), downstream queries can entirely skip reading irrelevant directories.

The Metastore Bridge: Unifying PySpark and Spark SQL



The Table Architecture Matrix: Managed vs. External

	Managed Tables	External Tables
Data Storage	Stored natively in Lakehouse 'Tables' area.	Stored in arbitrary external locations (e.g., 'Files' area).
Metadata Location	Spark Catalog.	Spark Catalog.
Deletion Impact	Deleting table DELETES underlying data files.	Deleting table leaves underlying data untouched.

Note: The preferred storage format for Fabric relational tables is Delta Lake (supporting ACID transactions and versioning).

PySpark API Wrapper

```
1 spark.sql("SELECT ProductID
2     FROM products")
3
```

Wraps standard SQL inside Python code. Excellent for passing query results directly into a PySpark variable (**bikes_df = spark.sql(...)**) for further programmatic manipulation.

Native Magic Command

```
1 %%sql
2 SELECT Category, COUNT...
```

Flips the entire notebook cell into native SQL mode. Bypasses Python entirely to query the catalog directly, instantly rendering formatted result tables in the UI.

Query data using the Spark SQL API

Use the Spark SQL API in code written in any language to query data in the catalog

```
# Return data from the products table as a dataframe using PySpark

bikes_df = spark.sql("SELECT ProductID, ProductName, ListPrice \
                      FROM products \
                      WHERE Category IN ('Mountain Bikes', 'Road Bikes')")

display(bikes_df)
```

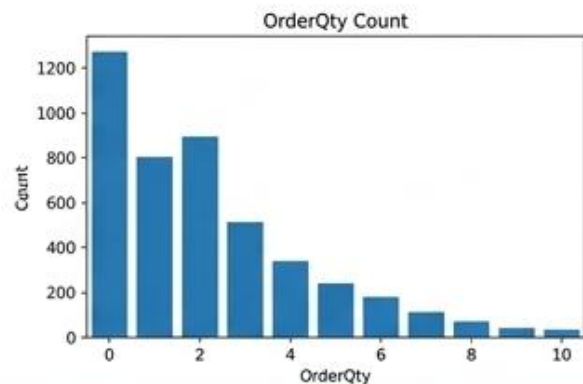
```
# Use %%sql magic to query objects in the catalog using native SQL

%%sql

SELECT Category, COUNT(ProductID) AS ProductCount
FROM products
GROUP BY Category
ORDER BY Category
```

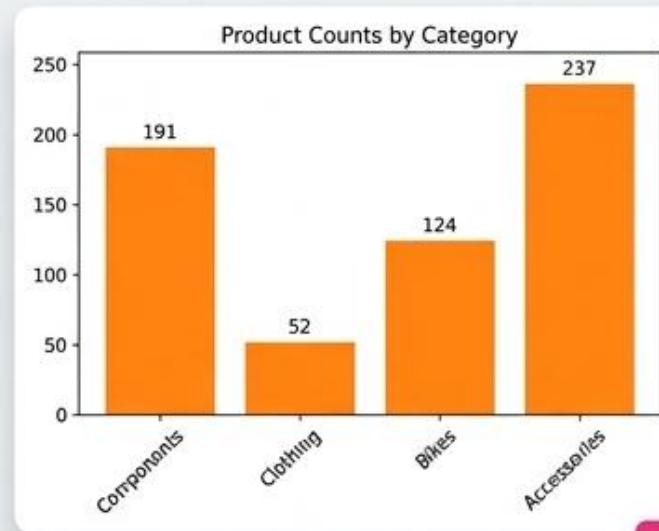
Built-in UI Charts

```
1 counts_df = df.select("ProductID", "OrderQty").groupBy("OrderQty").count()  
2 display(counts_df)
```



Zero-code profiling. Ideal for instant visual summaries directly beneath the executed SQL or PySpark cell.

Python Graphics (Matplotlib)



Maximum **customization** using Matplotlib or Seaborn. Requires converting the distributed Spark dataframe into a local Pandas dataframe first.

`.toPandas()`

Exercise

45 minutes



Lab 2: Analyze data with Apache Spark

Lab 2: Analyzing Data with Apache Spark in Microsoft Fabric



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-labs-michael-lively-ejlx/>

Part 4: Work with Delta Lake tables in Microsoft Fabric

<https://aka.ms/fabric-delta>



Learning objectives



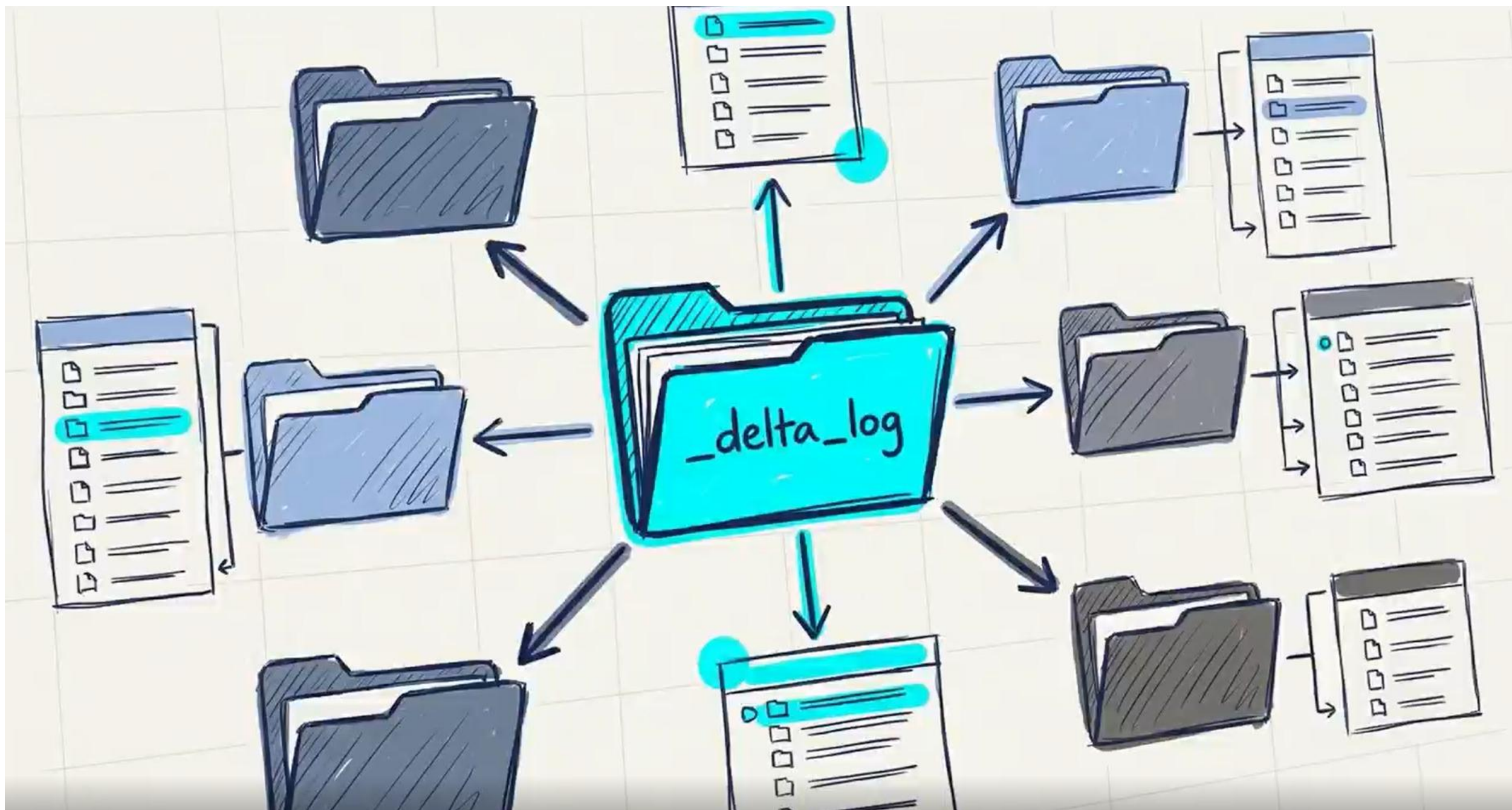
- Understand Delta Lake and delta tables in Microsoft Fabric
- Create and manage delta tables using Spark
- Optimize delta tables
- Use delta tables with Spark structured streaming

Part 4: Delta Lake Tables in Microsoft Fabric

RELATIONAL DATABASE CORE

CRUD Operations & Transactions

ID	Name	Category	Price
101	Telemetry_A	Sensor	\$15.00
102	Telemetry_B	Sensor	\$25.00
103	Legacy_Log	Archive	\$45.00



The Engine of the Lakehouse Architecture

Delta Lake is an open-source storage layer that brings relational database semantics directly to Spark-based data lakes.



ACID Transactions

Enforces atomicity, consistency, isolation, and durability via a serializable transaction log.



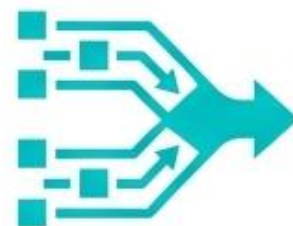
Relational CRUD

Enables standard SQL SELECT, INSERT, UPDATE, and DELETE operations directly on data lake files.



Time Travel

Data versioning enabled by immutable transaction logging, allowing queries on historical states.



Batch & Streaming

Unified processing engine. Delta tables act seamlessly as both sources and sinks for Spark Structured Streaming.

Instantiating Tables: Three Approaches

1. Dataframe Save (PySpark)

```
df.write.format("delta").saveAsTable("mytable")
```

(Note: Saves in Parquet format with a _delta_log folder)

2. Spark SQL

```
%sql
CREATE TABLE salesorders (
  Orderid INT NOT NULL,
  CustomerName STRING
) USING DELTA
```

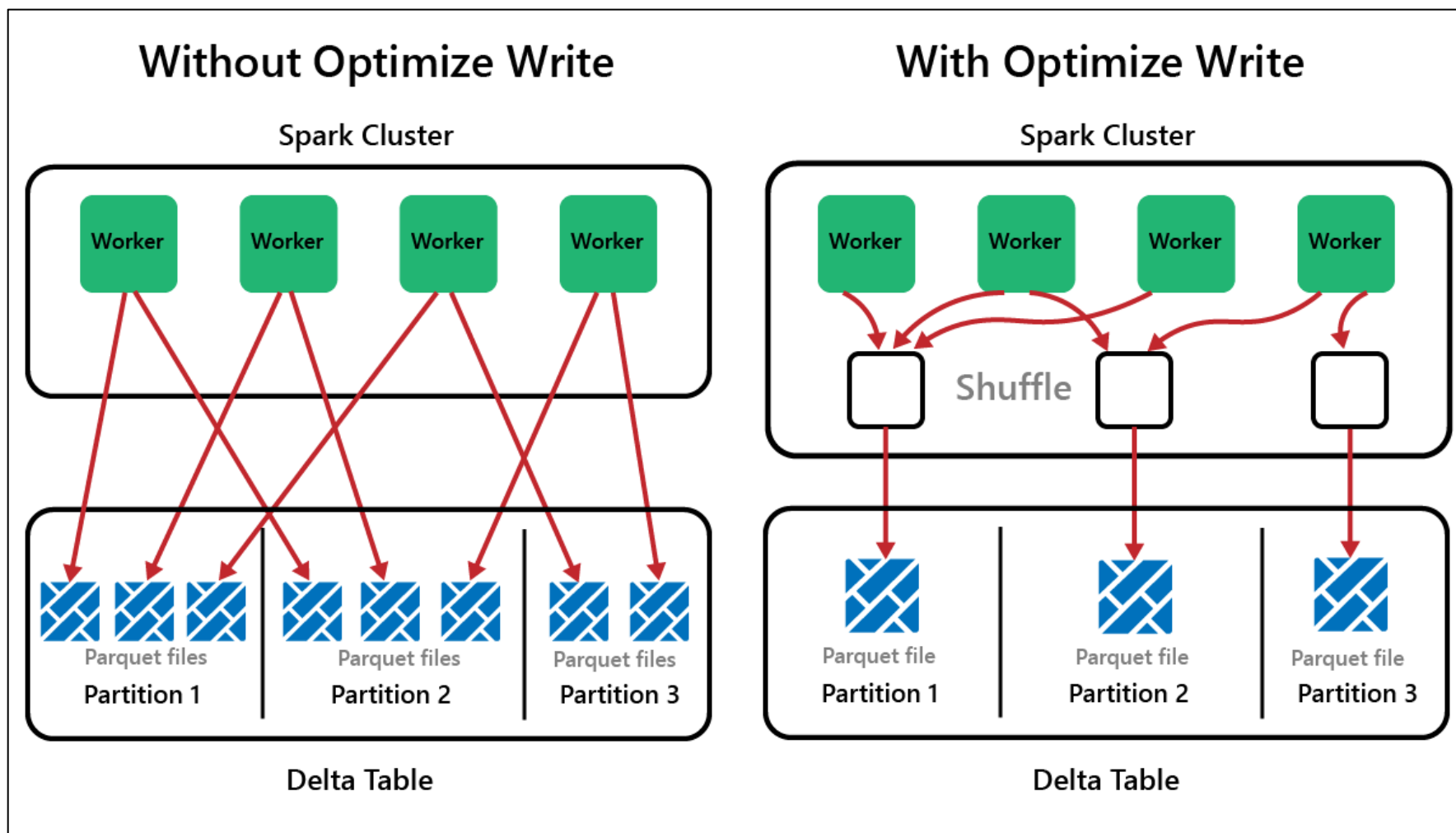
(Note: Can use LOCATION to map to existing Parquet folders)

3. DeltaTableBuilder API

```
DeltaTable.create(spark) \
  .tableName("products") \
  .addColumn("Productid", "INT") \
  .execute()
```

(Note: Creates metadata without populating files)

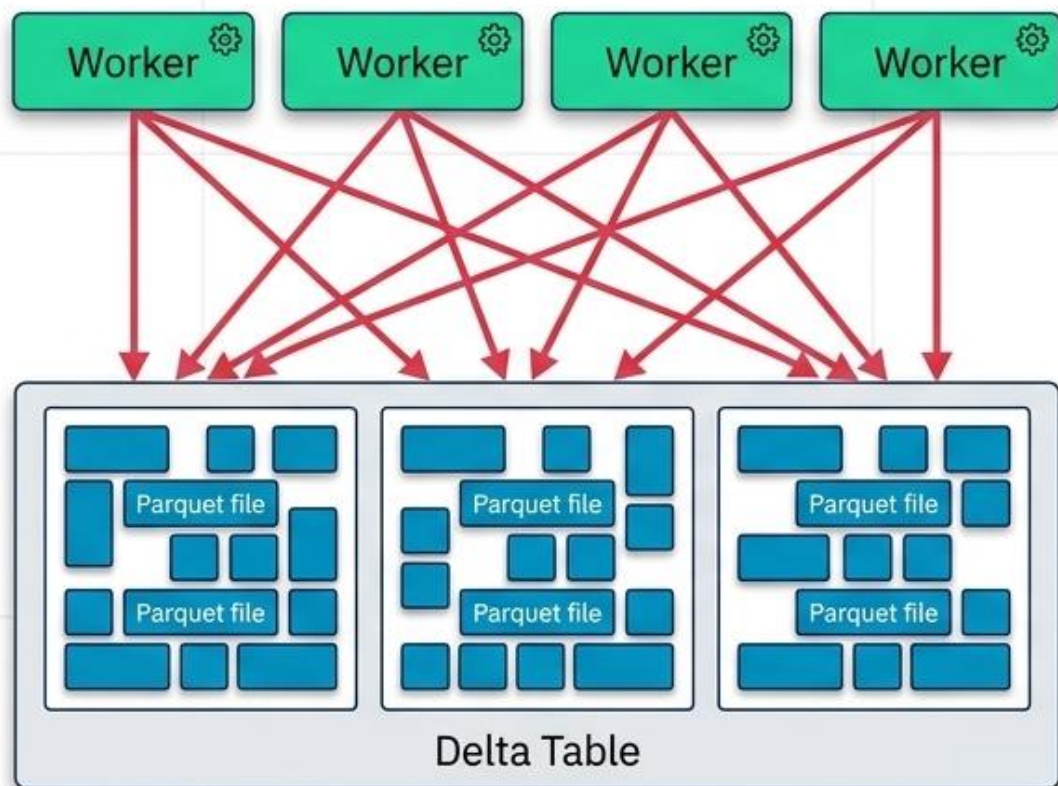
OptimizeWrite function



The “Small File Problem” & OptimizeWrite

Continuous parallel writes result in highly fragmented, tiny files that degrade query speed. OptimizeWrite shuffles data to write fewer, larger files.

Without Optimize Write



With Optimize Write

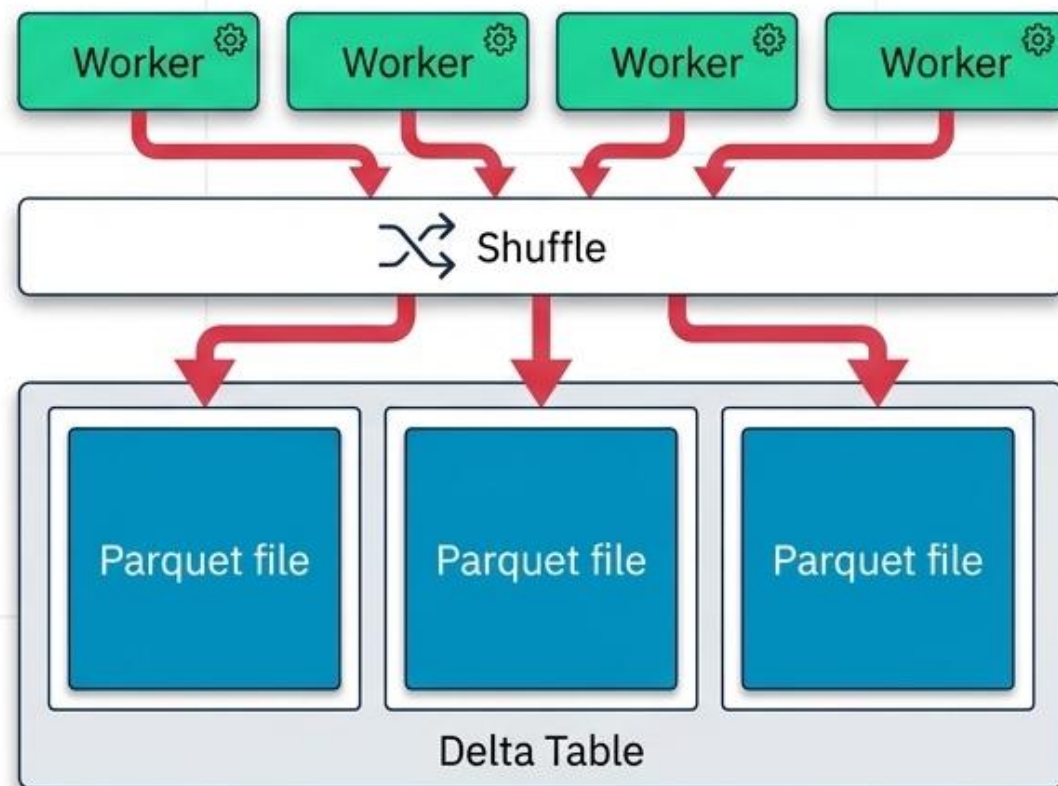


Table Maintenance: The Optimize Command

In Lakehouse Explorer → Table
... menu → Maintenance
→ Run OPTIMIZE

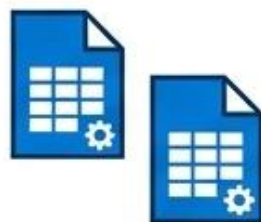
- ✓ - Fewer, larger files
- ✓ - Better compression
- ✓ - Efficient data distribution across nodes

Five small
Parquet files



Right-size
Parquet file

Two medium
Parquet files



Right-size
Parquet file

One right-size
Parquet file



Right-size
Parquet file

V-Order: Fabric's Secret Weapon

Write Impact



Read Performance



Microsoft Verti-Scan Technology

For Power BI and SQL engines in Fabric, V-Order enables lightning-fast, in-memory-like read times by applying custom sorting, row group distribution, and dictionary encoding while remaining 100% open-source Parquet compliant.

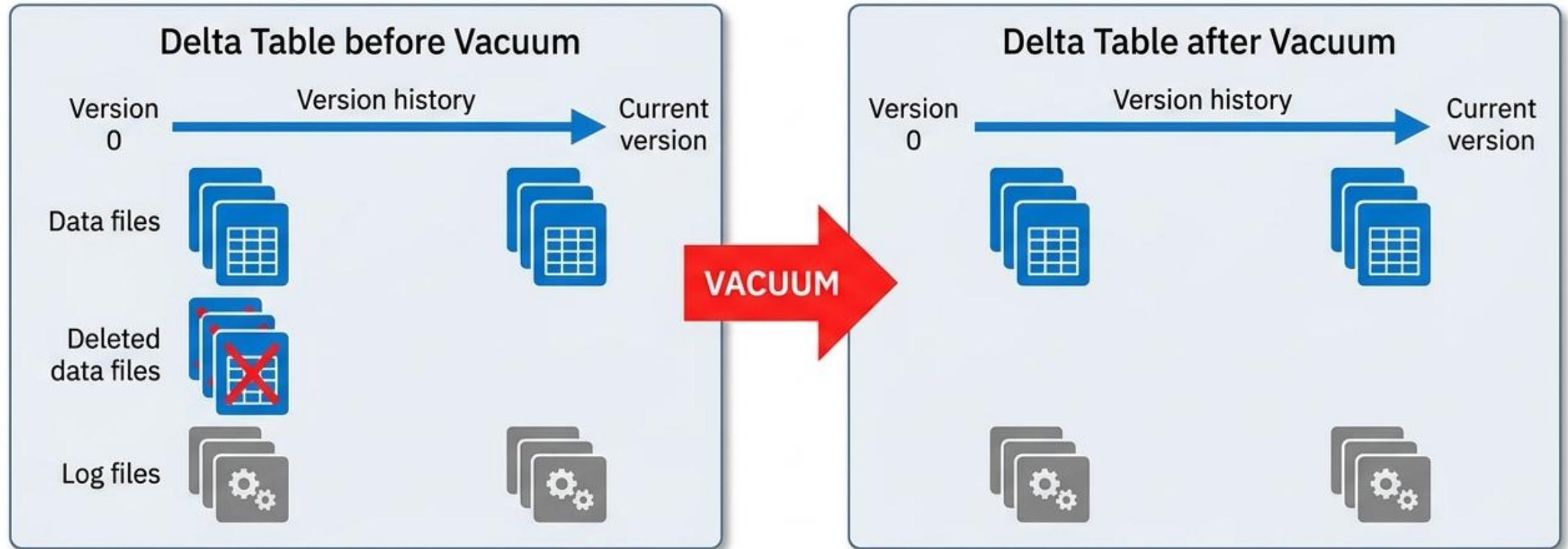


Diagnostic Tip: Disable V-Order for staging environments where data is write-heavy but read only once.

Managing State & Cost: The Vacuum Command

Fluent Data Playbook

Storage Bloat (unreferenced files)



Warning: Running **VACUUM** prevents Time Travel to points earlier than the retention period (Default: 168 hours).

Time Travel & Versioning

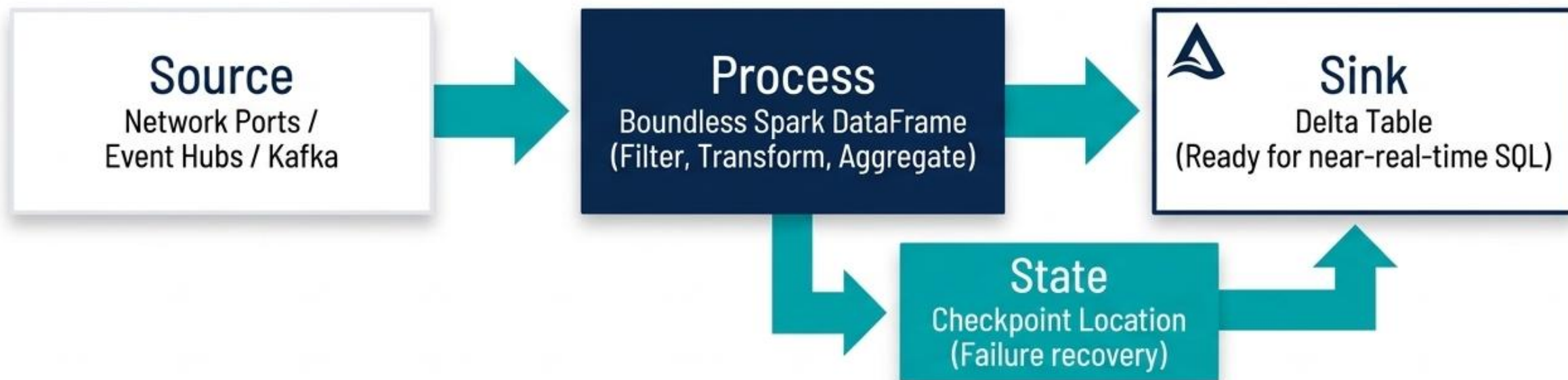


```
df.read.format('delta').option('versionAsOf', 0).load()  
# OR  
df.read.format('delta').option('timestampAsOf', '2023-04-04T20:04:23Z').load()
```

DESCRIBE HISTORY

Output: Mapped directly to timestamps, allowing you to instantly restore or query the exact state of your data at any prior moment.

Unifying Batch & Real-Time: Spark Structured Streaming



Spark processes batch and streaming data via the exact same API.
Delta tables can act as seamless sources or sinks for infinite data streams.

Streaming Implementation

1. Read Stream

```
stream_df = spark.readStream.format('delta')  
    .option('ignoreChanges', 'true')  
    .table('orders_in')
```

(Callout: Must use ignoreChanges or ignoreDeletes if source table undergoes modifications).

2. Transform

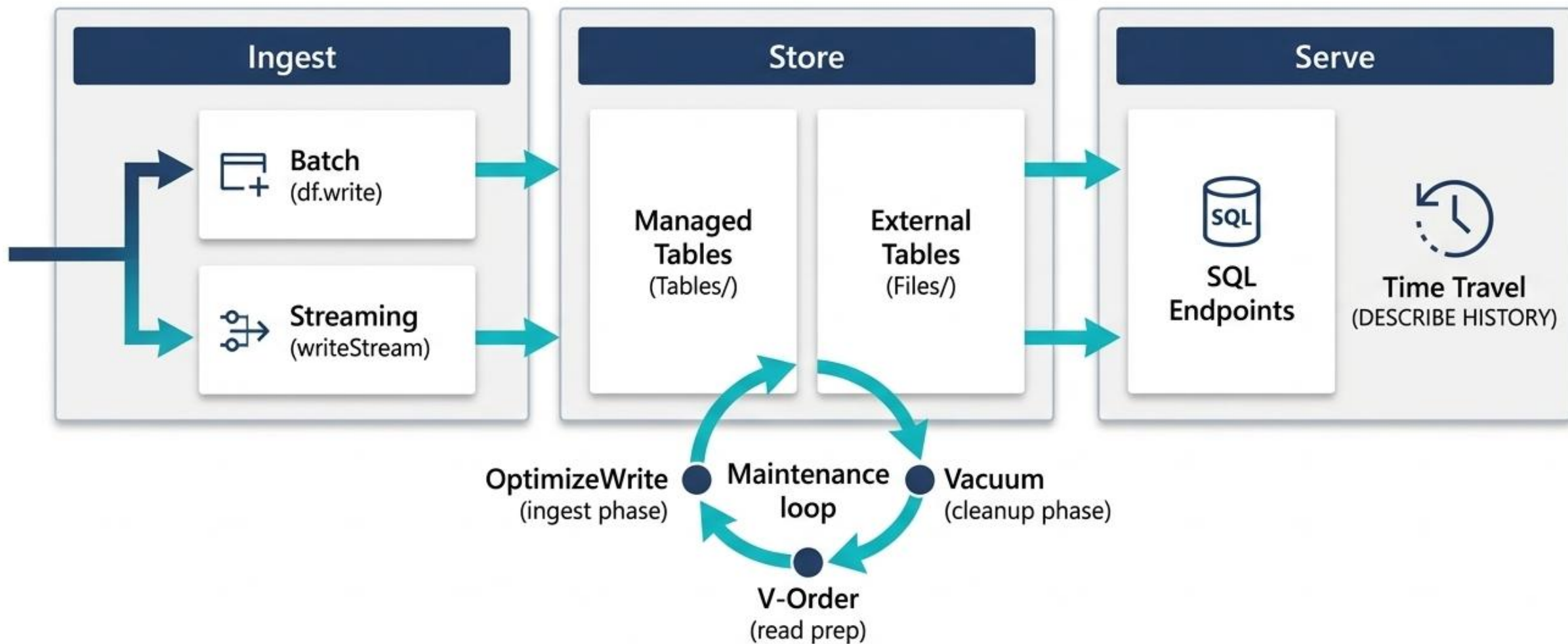
```
transformed_df = stream_df.filter(col('Price').isNotNull())
```

(Callout: checkpointLocation is mandatory to track stream state).

3. Write Stream

```
transformed_df.writeStream.format('delta')  
    .option('checkpointLocation', 'Files/checkpoint')  
    .start('Tables/processed')
```

Synthesis: The Delta Lake Blueprint in Fabric



One Unified Engine. Full ACID Compliance. Infinite Scalability.

Data versioning and time travel

Delta tables have transaction logs to “time travel” to previous versions of data.

```
# Use SQL to see the history of a table
```

version	timestamp	operation	operationParameters
2	2023-04-04T21:46:43Z	UPDATE	{"predicate":"(ProductId = 1)"}
1	2023-04-04T21:42:48Z	WRITE	{"mode":"Append","partitionBy":"[]"}
0	2023-04-04T20:04:23Z	CREATE TABLE	{"isManaged":"true","description":null,"partitionBy":"[]","properties":"{}"}

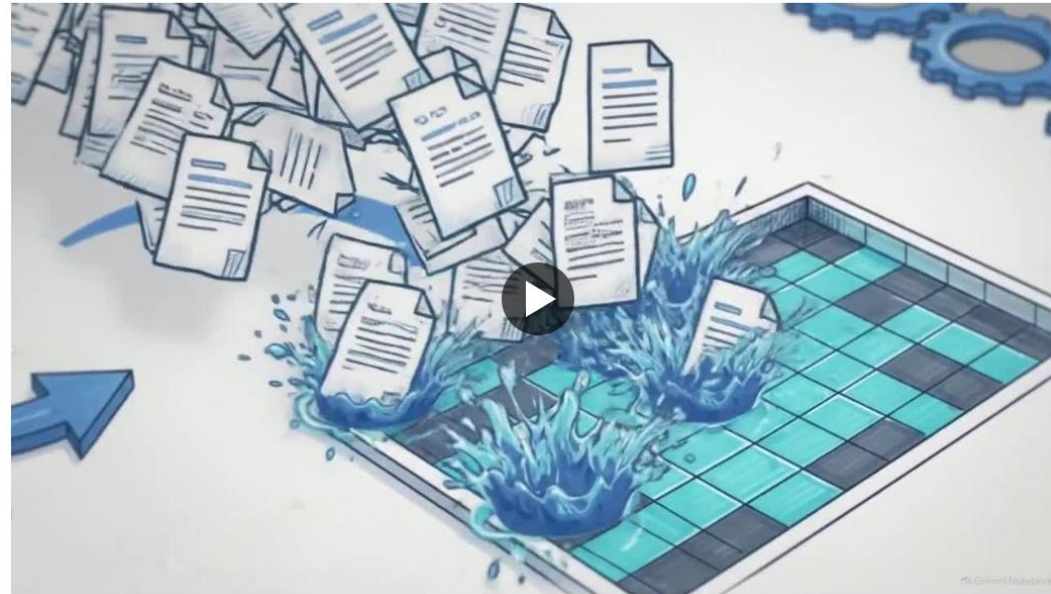
Exercise

45 minutes



Lab 3: Use delta tables in Apache Spark (fix)

Lab 3: Mastering Delta Tables in Microsoft Fabric



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-labs-michael-lively-ejlxc/>

Part 5: Ingest data with Dataflow Gen2 in Microsoft Fabric

<https://aka.ms/fabric-dataflow>



Learning objectives



- Describe Dataflow Gen2 capabilities in Microsoft Fabric
- Create Dataflow solutions to ingest and transform data
- Include a Dataflow in a pipeline

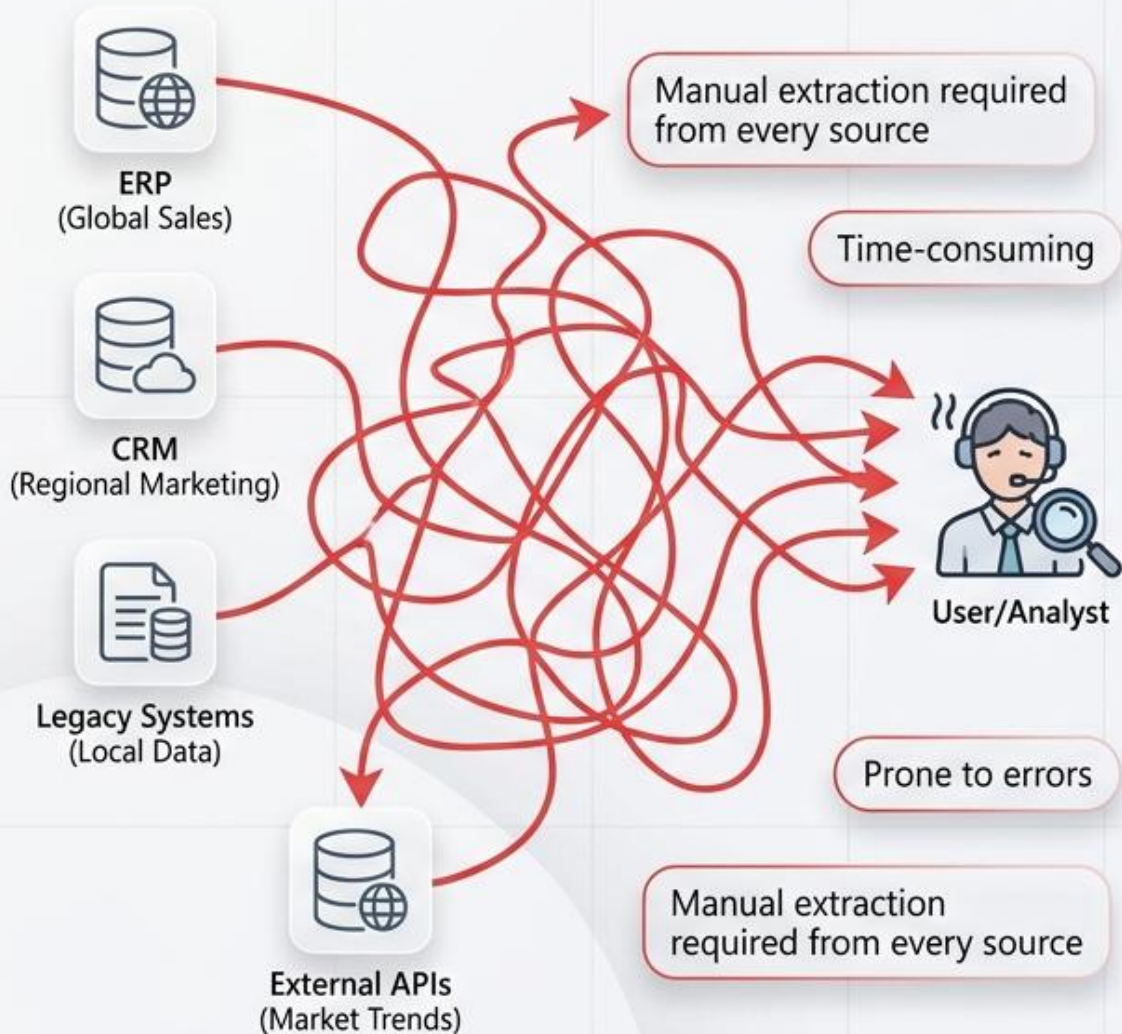
Part 5: Dataflows Gen2 in Microsoft Fabric



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>

The Data Integration Challenge

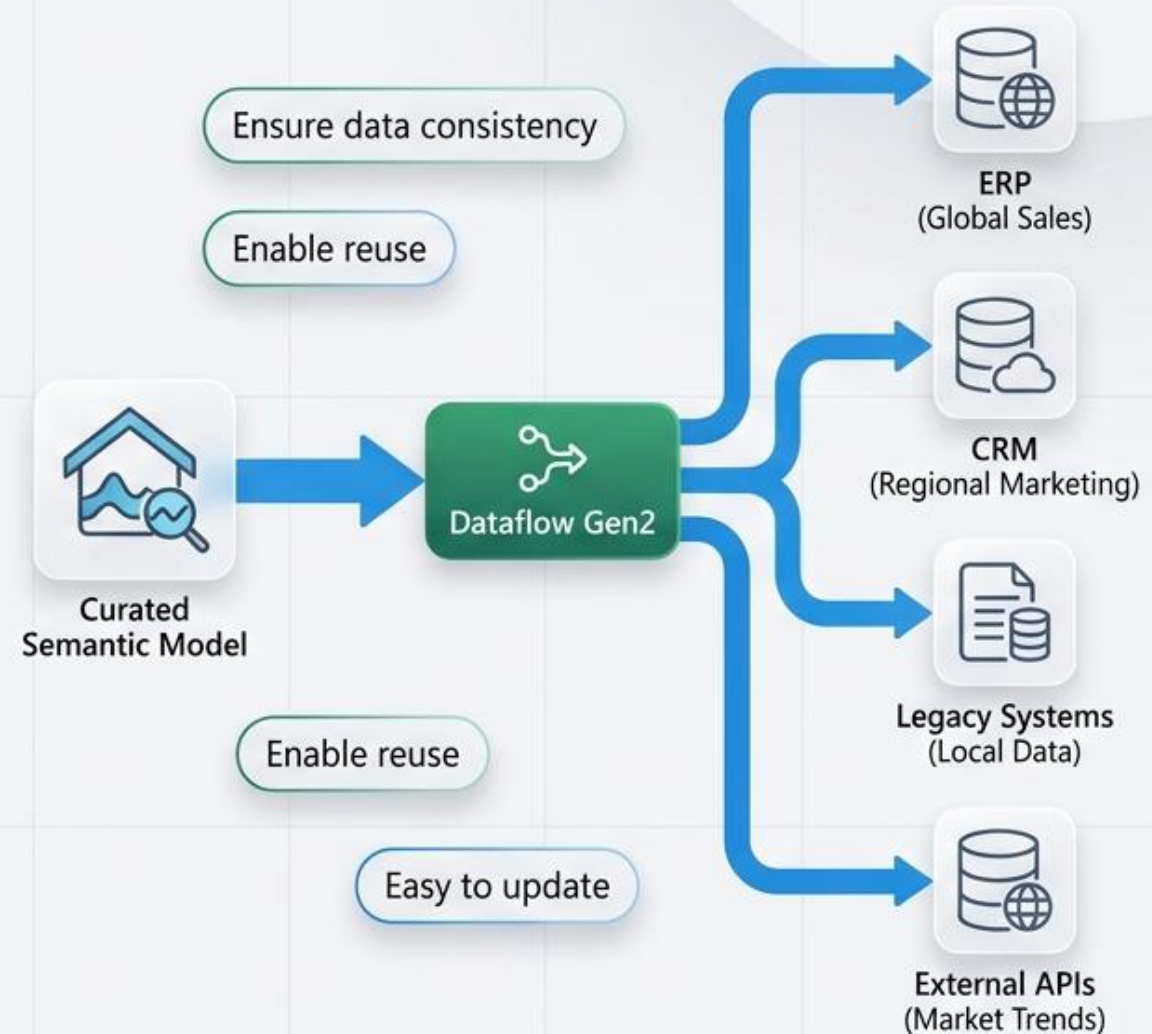
Before: Disconnected Data & Manual Effort



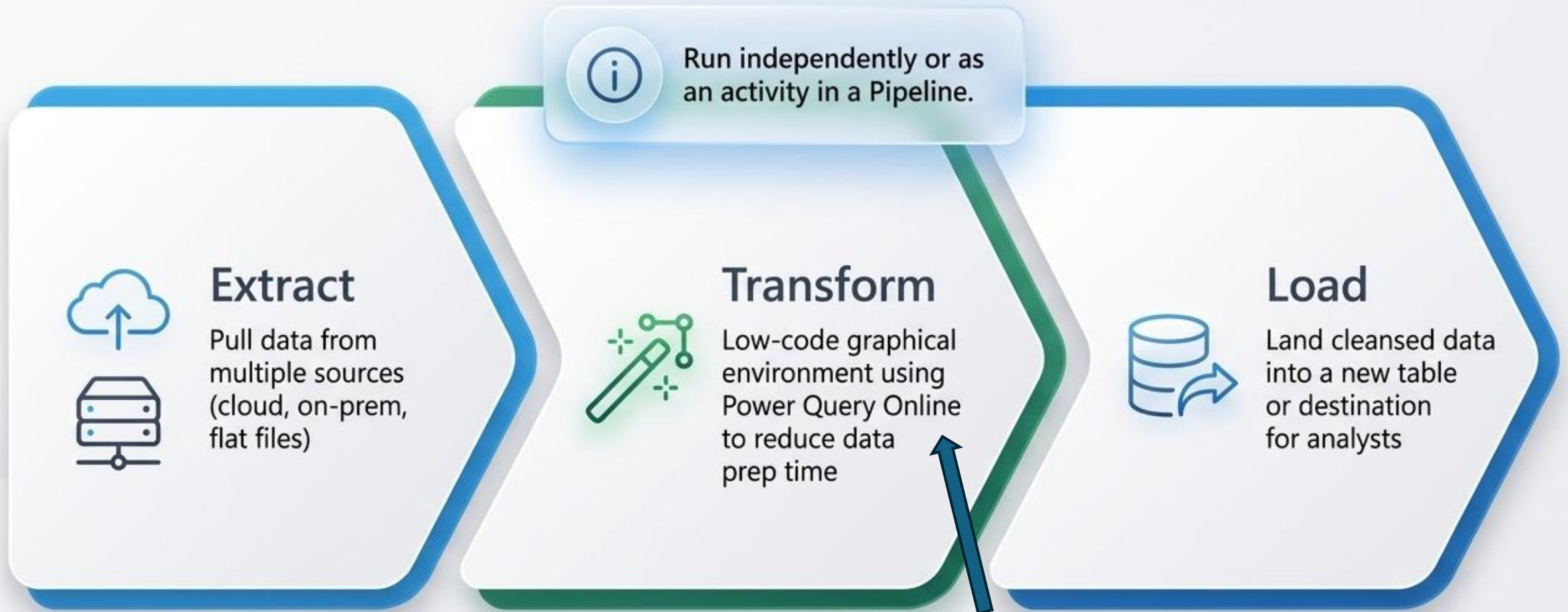
Before
&
After

Cloud Flow ETL

After: Fluent Data Fabric & Curated Flow



Introducing Dataflows Gen2



The Power Query Online Interface

1. Power Query Ribbon

The ribbon contains the following groups of commands:

- Home:** Get data, Enter data, Options, New query, Options, Parameters.
- Transform:** Refresh, Advanced editor, Manage, Query.
- Add column:** Add data destination, Manage columns, Reduce rows.
- View:** Choose columns, Remove columns, Keep rows, Remove rows, Filter rows, Sort, Transform, Combine, Map to entity, Export temp.
- Help:** Share.

2. Queries Pane

Shows a list of queries: orders, orders-2.

3. Diagram View

Visualizes the data flow between queries. 'orders-2' (3 steps) is connected to 'orders' (9 steps).

4. Data Preview Pane

Displays the first 15 rows of the 'orders' query. The formula bar shows: `Table.TransformColumnTypes(#"Changed column type 2", {"OrderDate", type`

	SalesOrderID	OrderDate	MonthNo	CustomerID	LineItem	ProductID
1	71774	6/1/2022, 12:00:00 AM	6	29847	1	836
2	71774	6/1/2022, 12:00:00 AM	6	29847	2	822
3	71776	6/1/2022, 12:00:00 AM	6	30072	1	907
4	71780	6/1/2022, 12:00:00 AM	6	30113	1	903
5	71780	6/1/2022, 12:00:00 AM	6	30113	2	983
6	71780	6/1/2022, 12:00:00 AM	6	30113	3	989
7	71780	6/1/2022, 12:00:00 AM	6	30113	4	748
8	71780	6/1/2022, 12:00:00 AM	6	30113	5	990
9	71780	6/1/2022, 12:00:00 AM	6	30113	6	926
10	71790	6/1/2022, 12:00:00 AM	6	30113	7	743
11	71780	6/1/2022, 12:00:00 AM	6	30113	8	782
12	71780	6/1/2022, 12:00:00 AM	6	30113	9	780
13	71780	6/1/2022, 12:00:00 AM	6	20113	10	737
14	71700	6/1/2022, 12:00:00 AM	6	20112	11	937
15	71700	6/1/2022, 12:00:00 AM	6	30113	12	AC7

Columns: 9 Rows: 99+

5. Query Settings Pane

Properties

- Name: orders
- Entity type: Custom

Applied steps

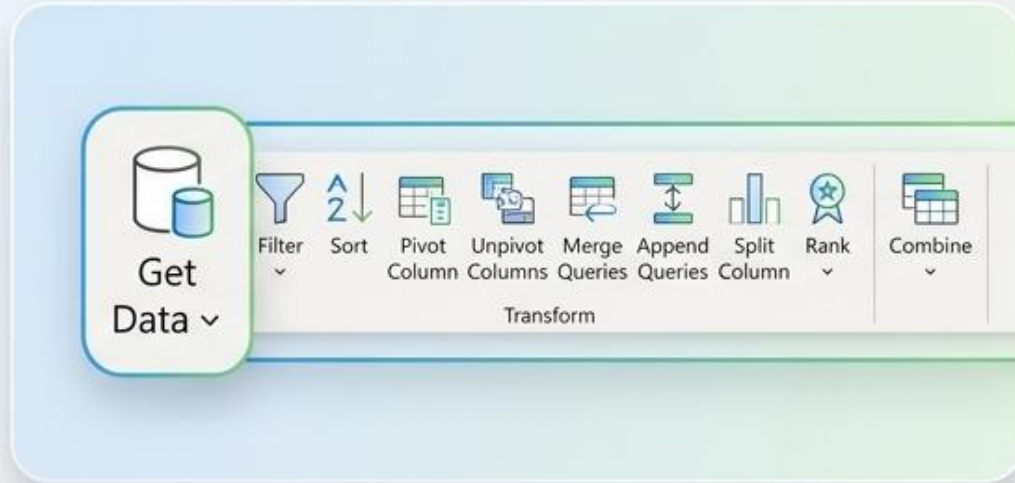
- Source
- Promoted h...
- Changed co...
- Appended ...
- Filtered rows
- Added cust...
- Reordered c...
- Changed co...
- Changed co...

Data destination

- Lakehouse

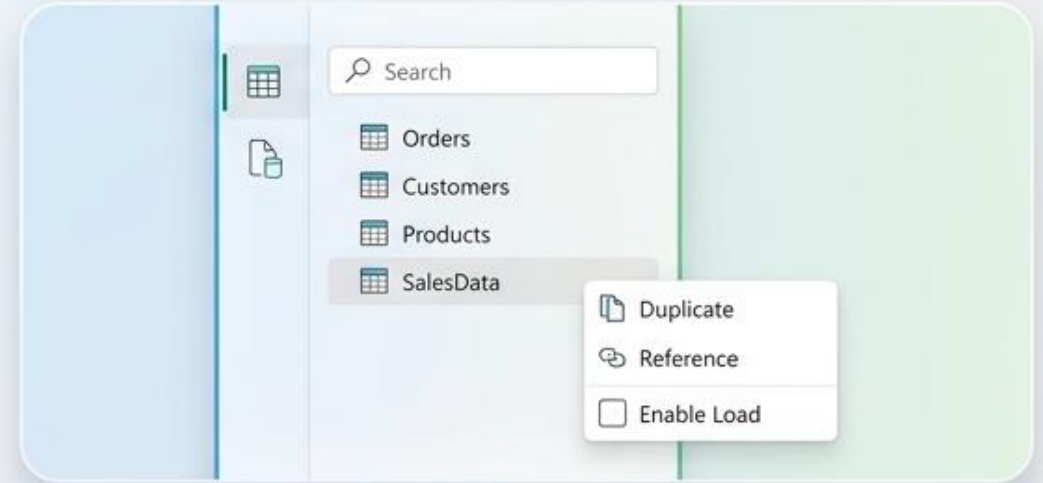
Deep Dive: Sourcing and Actions

1. Power Query Ribbon



- ▶ Connects to relational databases, Excel, SharePoint, Spark, and Fabric lakehouses
- ▶ Core Transformations: Filter/Sort, Pivot/Unpivot, Merge/Append, Split, Rank

2. Queries Pane



- ▶ Data sources act as “queries” (tables)
- ▶ Duplicate or reference a query (e.g., to create a star schema)
- ▶ Disable load for one-time imports

Deep Dive: Visualizing Logic and Data

3. Diagram View

Visually represents data sources and applied transformations (shapes connected by lines), making complex lineage clear.

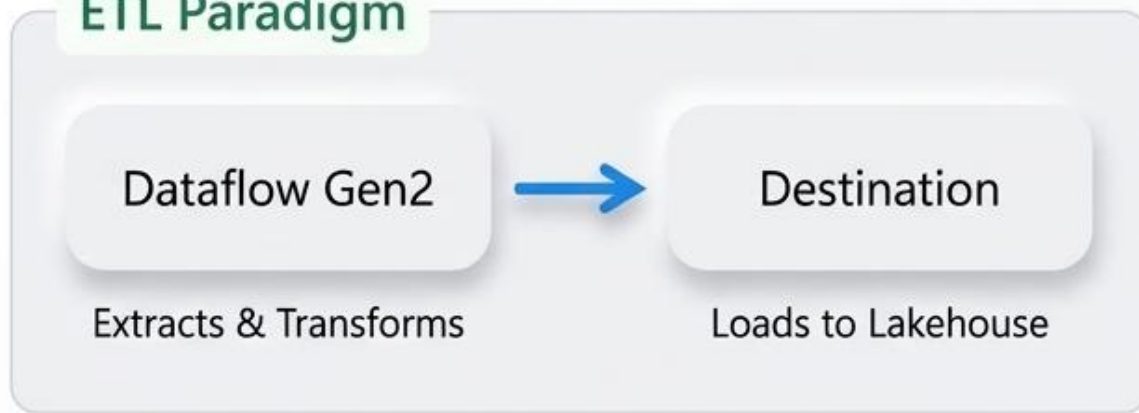


4. Data Preview Pane

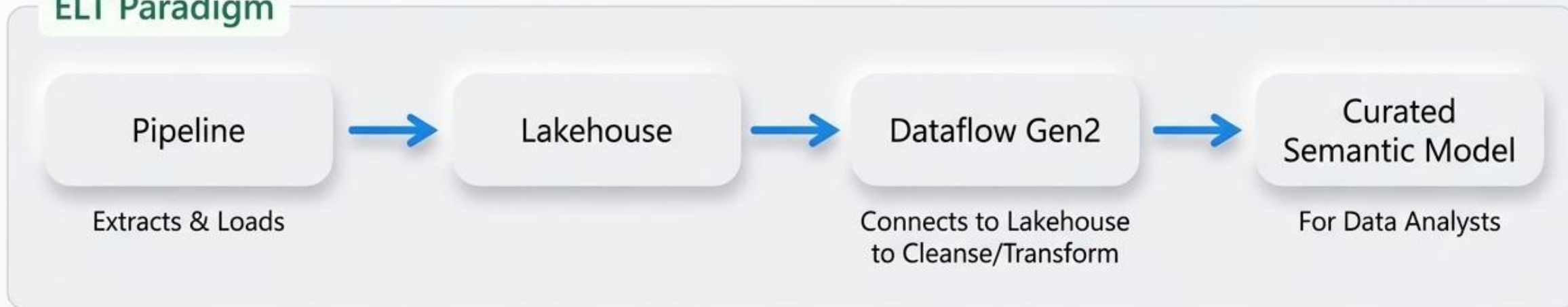
Shows a subset of data to test transformations in real-time. Drag and drop columns, right-click to filter.

Architectural Synthesis: ETL vs. ELT

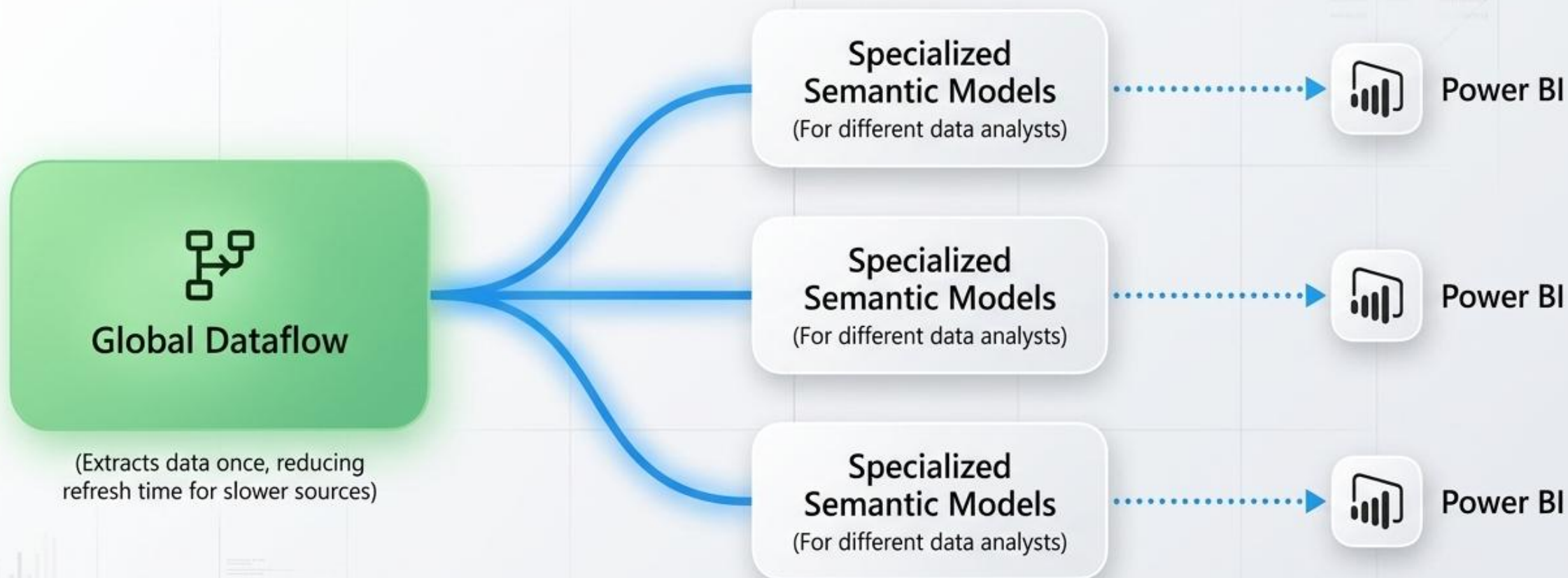
ETL Paradigm



ELT Paradigm



The Power of Reusability at Scale



Discoverability: Making dataflows discoverable allows analysts to connect directly via Power BI Desktop, drastically reducing report data prep time without recreating source connections.

Diagnostic Matrix: Strategic Fit

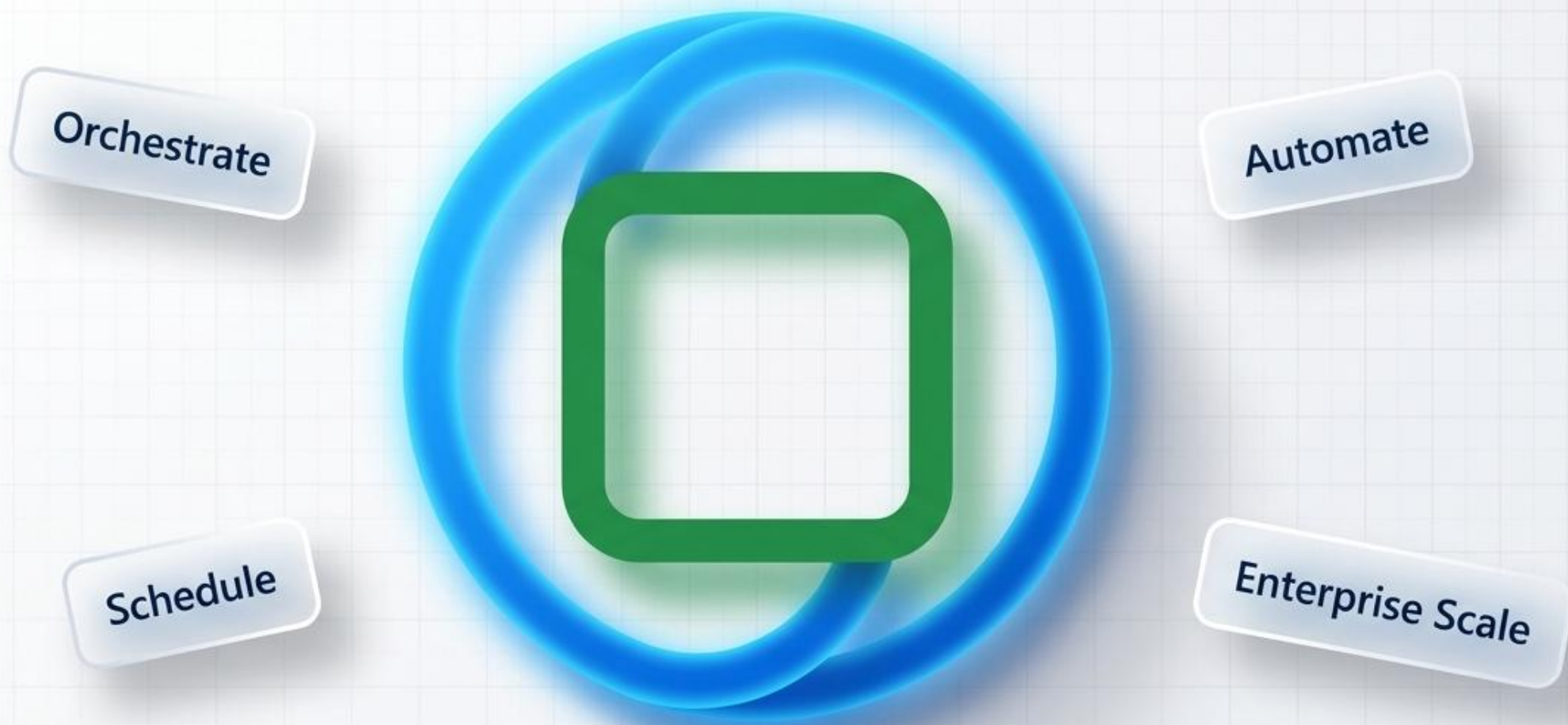
Benefits

- ✓ Extend data with standard dimensions (e.g., date dim)
- ✓ Allow self-service access to subset of DW
- ✓ Optimize performance by extracting once
- ✓ Simplify data source complexity for analysts
- ✓ Ensure low-code consistency and quality

Limitations

- ⚠ Not a complete data warehouse replacement
- ⚠ Does not support row-level security
- ⚠ Strictly requires a Fabric capacity workspace

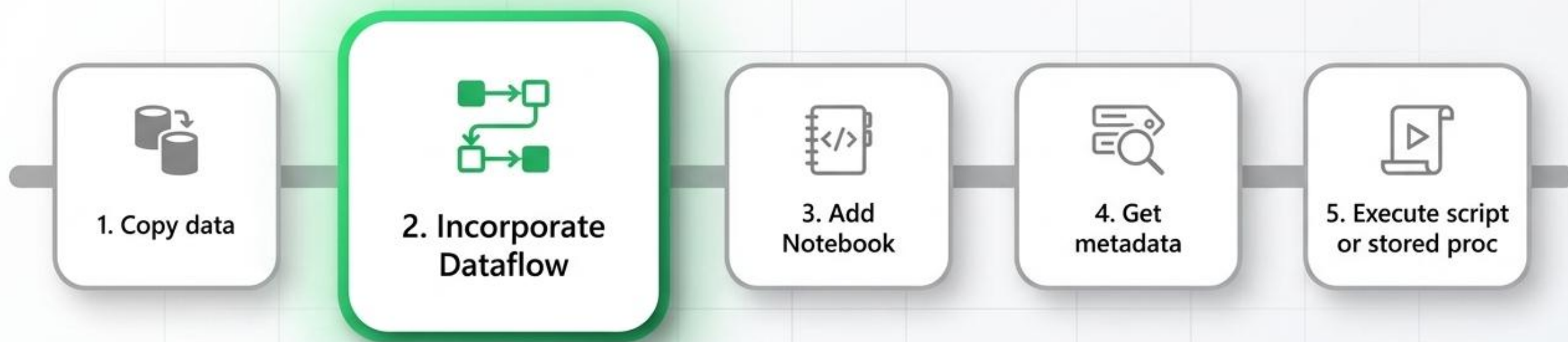
M-Code



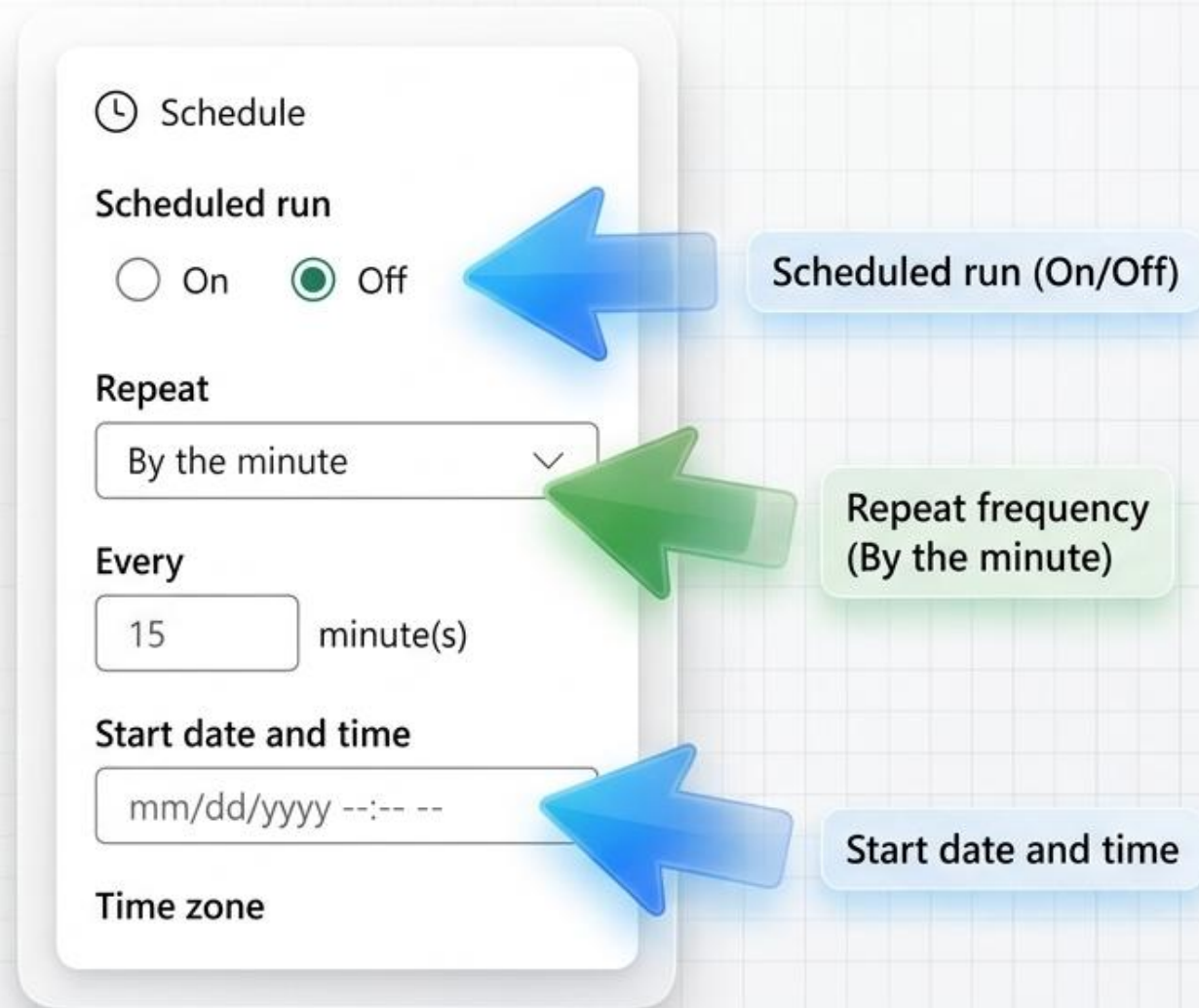
Elevating with Pipelines

Add a Dataflow Gen2 activity to your orchestration

Pipeline Orchestration Activities



Automation and Scheduling



The diagram shows a scheduling configuration panel on the left with three callout boxes on the right. A blue arrow points from the 'Scheduled run (On/Off)' callout to the 'On/Off' radio buttons. A green arrow points from the 'Repeat frequency (By the minute)' callout to the 'By the minute' dropdown menu. A blue arrow points from the 'Start date and time' callout to the date/time input field.

🕒 Schedule

Scheduled run

☐ On ☒ Off

Repeat

By the minute ▾

Every

minute(s)

Start date and time

Time zone

Scheduled run (On/Off)

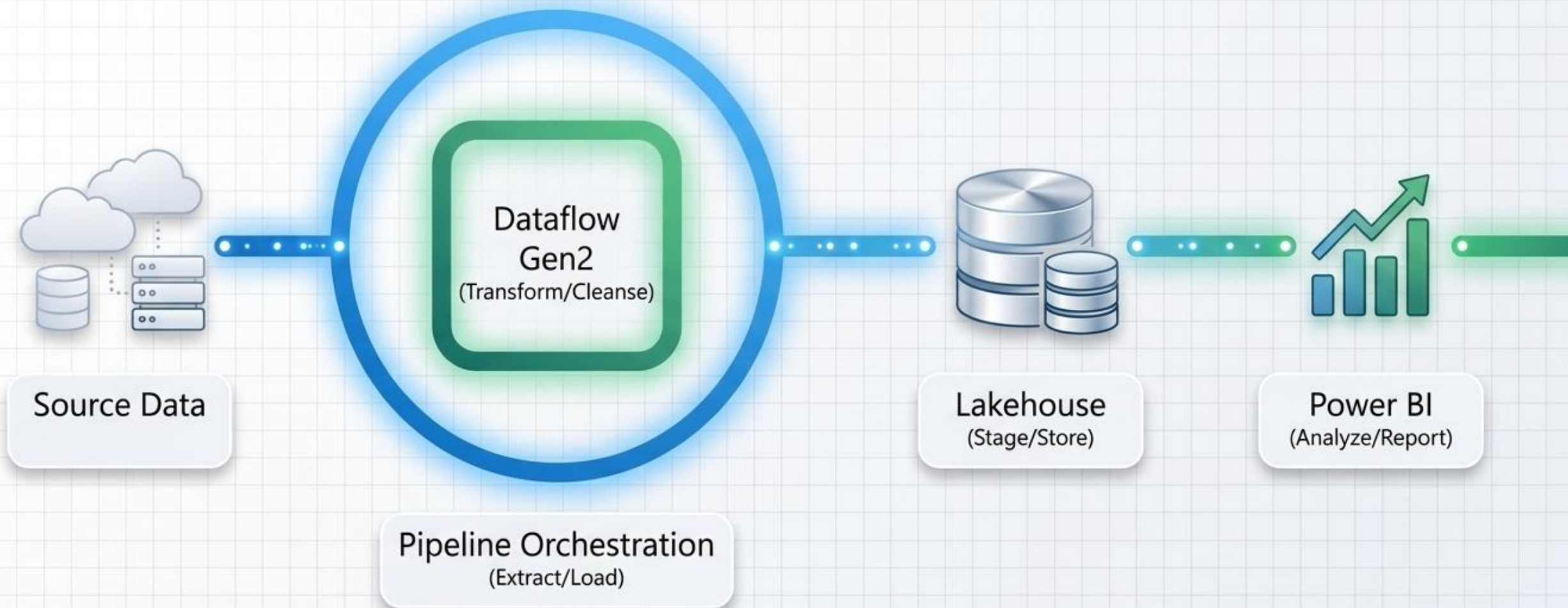
Repeat frequency (By the minute)

Start date and time

Replace manual runs with automation.

Pipelines can be activated by triggers or scheduled to refresh enterprise/ frequently changing data automatically, allowing engineers to focus on other tasks.

The End-to-End Analytics Lifecycle



Exercise

30 minutes



Lab 4: Create and use a Dataflow Gen2 in Microsoft Fabric

Lab 4: Microsoft Fabric Data Ingestion Pipelines



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-labs-michael-lively-ejlx/>

Part 6: Orchestrate processes and data movement with Microsoft Fabric

<https://aka.ms/fabric-pipeline>

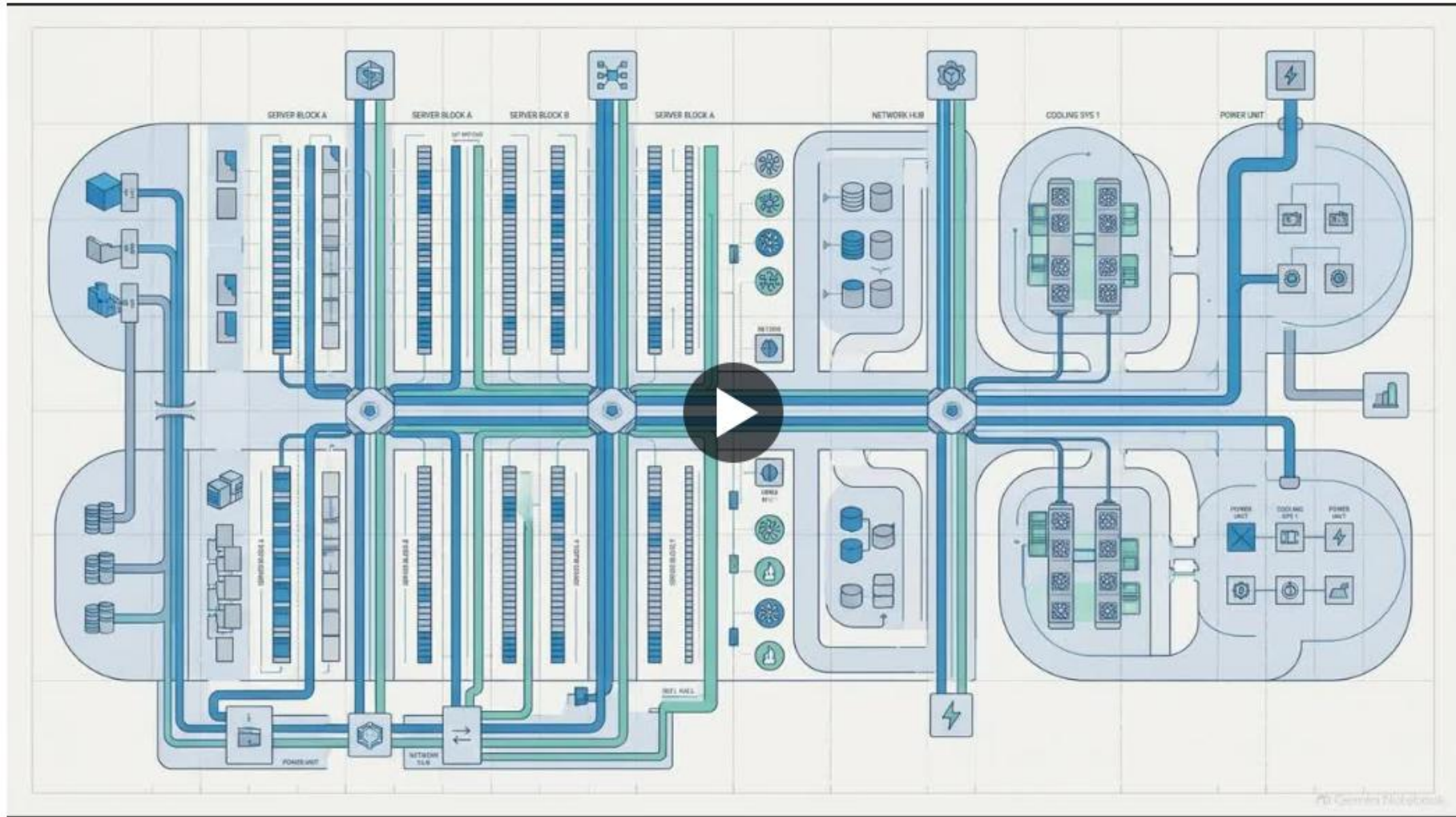


Learning objectives



- Describe pipeline capabilities in Microsoft Fabric.
- Use the Copy Data activity in a pipeline.
- Create pipelines based on predefined templates.
- Run and monitor pipelines.

Part 6: Microsoft Fabric Data Pipeline Orchestration



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-michael-lively-wvtqc/>

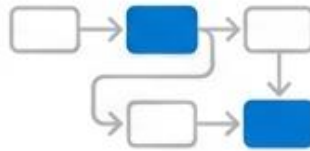
Automating Extract, Transform, and Load (ETL) Processes

Operational Data Stores



Microsoft Fabric Pipeline

A defined sequence of activities orchestrating complex data movement and processing.



Analytical Data Stores



Lakehouse

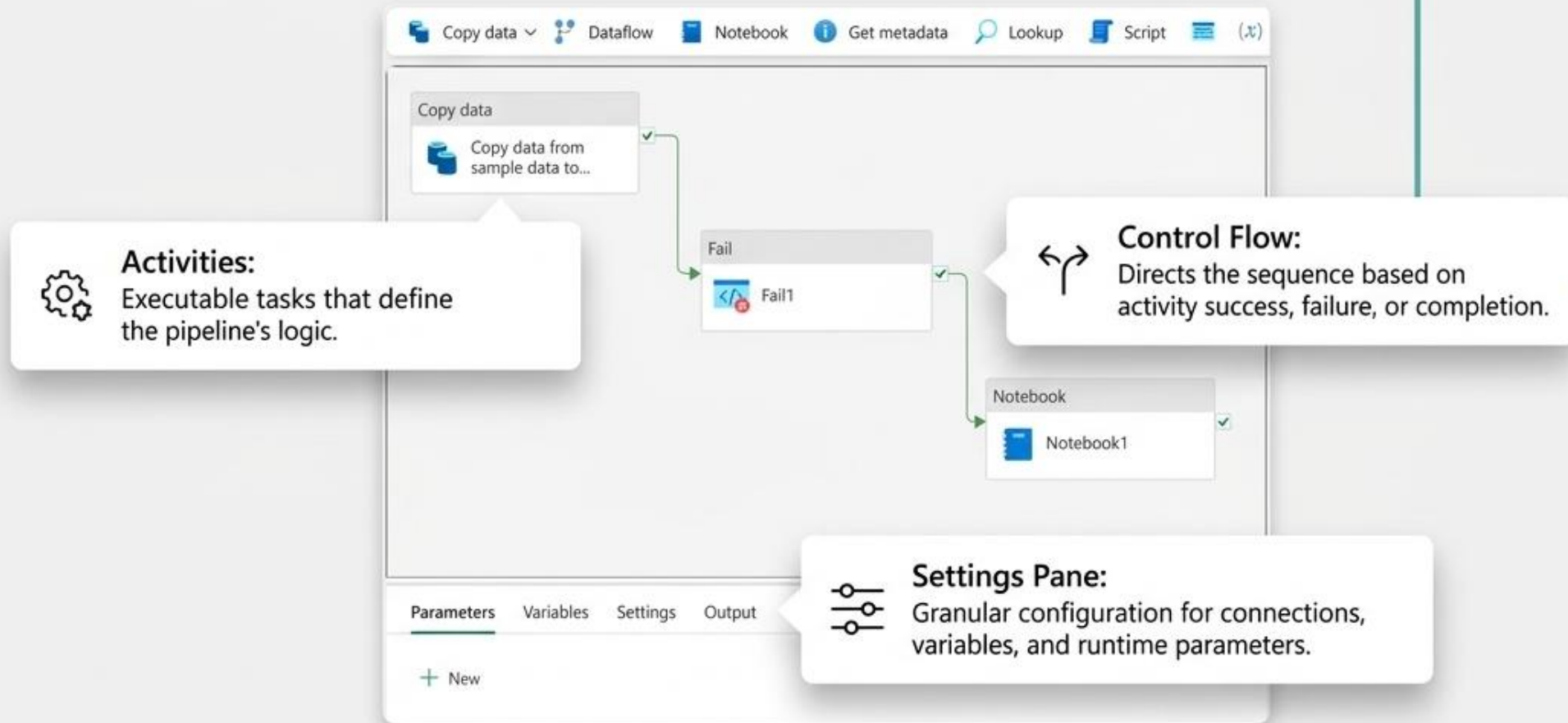


Data Warehouse



SQL Database

The Anatomy of the Pipeline Canvas



The graphical interface enables complex branching and looping logic with minimal to no coding.

The Three Pillars of Pipeline Activities



Data Movement

Extracts data from external sources and loads it into a destination.

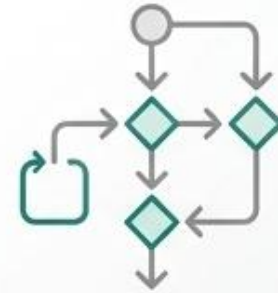
Example: Copy Data Activity



Data Transformation

Processes and shapes data mid-flight.

Examples: Dataflow Gen2, Notebooks, Stored Procedures



Control Flow

Orchestrates the ingestion and transformation flow.

Examples: Loops, conditional branching, parameters

Driving Reusability at Scale with Parameters



Hardcoded Values

Inflexible execution. Requires building duplicate pipelines for different targets, locking data to fixed locations (e.g., "Folder_A").

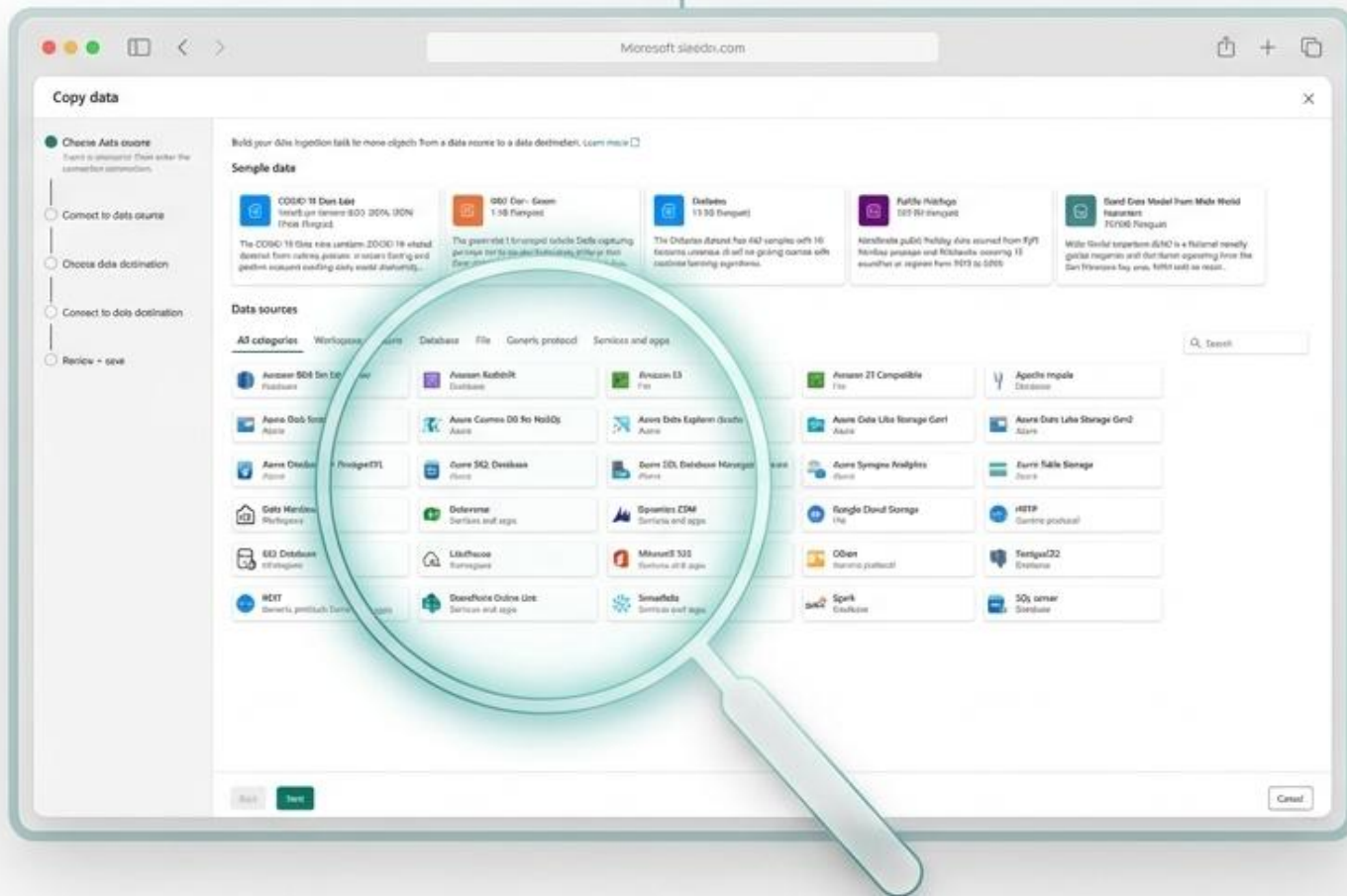


Dynamic Execution

Specific values are passed at runtime, creating flexible processes (e.g., dynamically specifying the target folder name during execution).

Parameters exponentially increase the reusability of ingestion and transformation processes across varying environments

Deep Dive: The Copy Data Engine Ecosystem



Key Capabilities

- ✓ Moves data between supported stores prior to heavy transformation.
- ✓ Natively supports OneLake destinations (Lakehouse, Warehouse, SQL DB).
- ✓ Ingests from almost any external operational data source.

Configuring and Sequencing Data Movement

The screenshot displays the Data Factory 'Copy data' activity configuration page. The interface includes a top navigation bar with tabs for Home, Activities, Run, and View. Below this is a toolbar with icons for Validate, Run, Schedule, View run history, Copy data, Dataflow, Notebook, Lookup, and Invoke pipeline. The main content area is divided into two sections: 'Select Activity' and 'Configure Settings'. The 'Select Activity' section shows a list of activities, with 'Copy_Product_Data' selected. The 'Configure Settings' section is further divided into tabs: General, Source, Destination, Mapping, and Settings. The 'Destination' tab is active, showing configuration options for the data store type (Workspace or External), the workspace data store type (Lakehouse), the Lakehouse name (DP601_Bronze), the root folder (Tables or Files), the table name (WWI_Products), and the table type (Edit). A 'Preview data' button is also visible.

1 Select Activity

2 Configure Settings

Repeatable Ingestion Pattern



[Delete Data]
Remove existing



[Copy Data]
Replace from external



[Notebook]
Transform and load

Architectural Decision Matrix: Choosing an Ingestion Tool

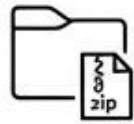
Tool	Use Case	Strengths
 Copy Job	Simplified experience for copy-only workloads.	Bulk, incremental, or Change Data Capture (CDC).
 Copy Data Activity	Moving data as a single step within a broader pipeline orchestration.	Detailed control over the copy operation; allows downstream transformation.
 Dataflow Gen2 Activity	Complex pre-processing requirements.	Leveraging Power Query to transform or combine data from multiple sources.

Accelerating Development with Pipeline Templates



Predefined Templates

Microsoft-authored solutions for common scenarios, accessible via the Templates tile.



Custom Templates

Export custom pipelines as .zip files and import them to reuse architecture.

Custom templates enforce consistent pipeline structures across workspaces while isolating environment-specific connections.

Templates Gallery



Bulk Copy from Database

by Microsoft

Use this template to copy data in bulk from a database using an external control table to store the partition list of your source tables....



Bulk Copy from Files to Database

by Microsoft

Use this template to copy data in bulk from Azure Data Lake Storage Gen2 to Azure SQL Database.



Copy data from ADLS Gen2 to Lakehouse file

by Microsoft

Use this template to copy data from ADLS Gen2 to a specified file location in your Lakehouse.



Copy data from ADLS Gen2 to Lakehouse Table

by Microsoft

Use this template to copy data from ADLS Gen2 to a specified table in your Lakehouse.



Copy data from Azure SQL DB to Lakehouse Table

by Microsoft

Use this template to copy data from your Azure SQL database to a specified table in your Lakehouse.



Copy data from sample data to Lakehouse file

by Microsoft

Use this template to copy data from sample data (NYC Taxi - Green) to Lakehouse file.



Copy data from sample data to Warehouse

by Microsoft

Use this template to copy data from sample data (Retail Data Model from Wide World Importers) to your Fabric Warehouse....



Copy multiple files containers between File Stores

by Microsoft

Use this template to leverage multiple copy activities to copy containers or folders between file based stores, where each copy...

Use templates for common activities



Bulk Copy from Database

by Microsoft

Use this template to copy data in bulk from a database using an external control table to store the partition list of your source tables....



Bulk Copy from Files to Database

by Microsoft

Use this template to copy data in bulk from Azure Data Lake Storage Gen2 to Azure SQL Database.

...



Copy data from ADLS Gen2 to Lakehouse file

by Microsoft

Use this template to copy data from ADLS Gen2 to a specified file location in your Lakehouse.

...



Copy data from ADLS Gen2 to Lakehouse Table

by Microsoft

Use this template to copy data from ADLS Gen2 to a specified table in your Lakehouse.

...



Copy data from Azure SQL DB to Lakehouse Table

by Microsoft

Use this template to copy data from your Azure SQL database to a specified table in your Lakehouse.

...



Copy data from sample data to Lakehouse file

by Microsoft

Use this template to copy data from sample data (NYC Taxi - Green) to Lakehouse file.

...



Copy data from sample data to Warehouse

by Microsoft

Use this template to copy data from sample data (Retail Data Model from Wide World Importers) to your Fabric Warehouse....



Copy multiple files containers between File Stores

by Microsoft

Use this template to leverage multiple copy activities to copy containers or folders between file based stores, where each copy...

Operational Execution: Triggering a Pipeline Run



On-Demand

Immediate manual execution (requires saving prior changes).



Scheduled

Defined start/end dates, frequency, and time zone. Schedule parameters must match pipeline parameters.



Event-Based

Triggered automatically by a file arriving in OneLake or Azure Blob Storage.

Initiate
Pipeline Run

Visibility and Control: The Run History

Action Ribbon:
Validate configuration
before running.

The screenshot shows the Gemini Notebook interface. The top navigation bar includes 'Home', 'Activities', 'Run', and 'View'. The 'Run' tab is active, showing an 'Action Ribbon' with buttons: 'Validate' (checked), 'Run', 'Schedule', and 'View run history'. A red box highlights the 'Validate' button. Below the ribbon, a 'Copy data' dialog box is visible. On the right, a 'Run History' table is displayed with columns: 'Name', 'Status', and 'Start time'. A red box highlights the 'Status' column, which contains a green checkmark icon. The table lists ten runs, all of which are 'Completed' except for one that is 'Failed'.

Name	Status	Start time
Process rides pipeline	Completed	3:17 PM, 6/21/23
Process rides pipeline	Failed	2:31 PM, 6/28/23
Process rides pipeline	Completed	2:33 PM, 6/28/23
Process rides pipeline	Completed	2:44 PM, 6/28/23
Process rides pipeline	Completed	2:44 PM, 6/28/23
Process rides pipeline	Completed	2:50 PM, 6/28/23
Process rides pipeline	Completed	3:41 PM, 6/28/23
Process rides pipeline	Completed	3:33 PM, 6/30/23
Process rides pipeline	Completed	3:55 PM, 6/30/23

Status Tracking:
Monitor execution
success in real-time.



Trace

Use the unique **Run ID** to drill down into activity durations, inputs, outputs, and specific error details.



Recover

When a run fails, correct the underlying issue and trigger a rerun strictly from the point of the failed activity.

Advanced Telemetry & Workspace Troubleshooting

The Gantt View



Visualizing Workloads:

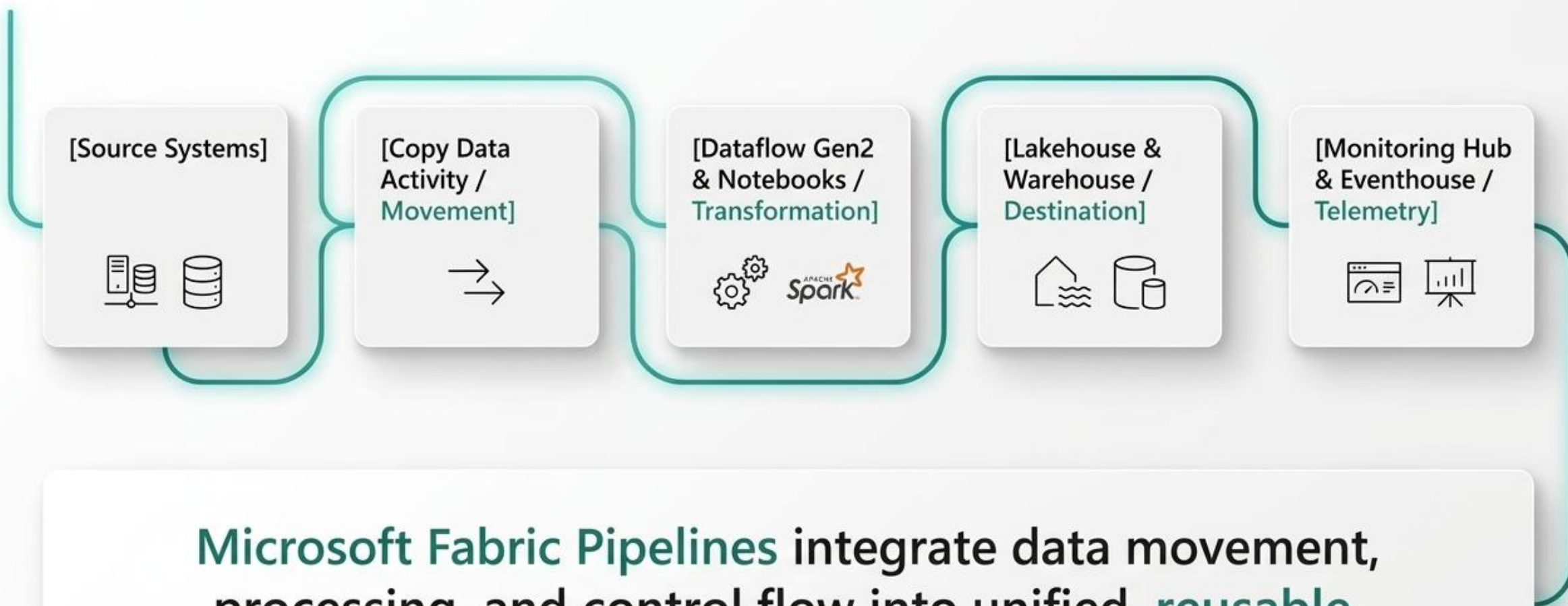
Compare run durations across the workspace to easily identify bottlenecks, delays, or overlapping runs.

Workspace Monitoring via KQL

```
// Workspace Execution Logs
WorkspaceLogs
| where EventType == "Execution"
| where Status in ("Failed", "Completed")
| summarize Duration = max(EndTime) - min(StartTime), ErrorCount =
  countif(Status == "Failed") by PipelineName, RunId
| order by Duration desc
| take 10
```

Log-Level Analysis: Enable workspace monitoring to store execution logs in an eventhouse, allowing complex querying using KQL.

End-to-End Orchestration Architecture



Microsoft Fabric Pipelines integrate data movement, processing, and control flow into unified, **reusable**, and **deeply monitored** enterprise architectures.

Theory to Practice: Begin Orchestration

 30 Minutes

- ✓ Create a pipeline and utilize the Copy Data activity to move an external source into a Lakehouse.
- ✓ Enhance the orchestration by adding transformation activities to process the ingested data.



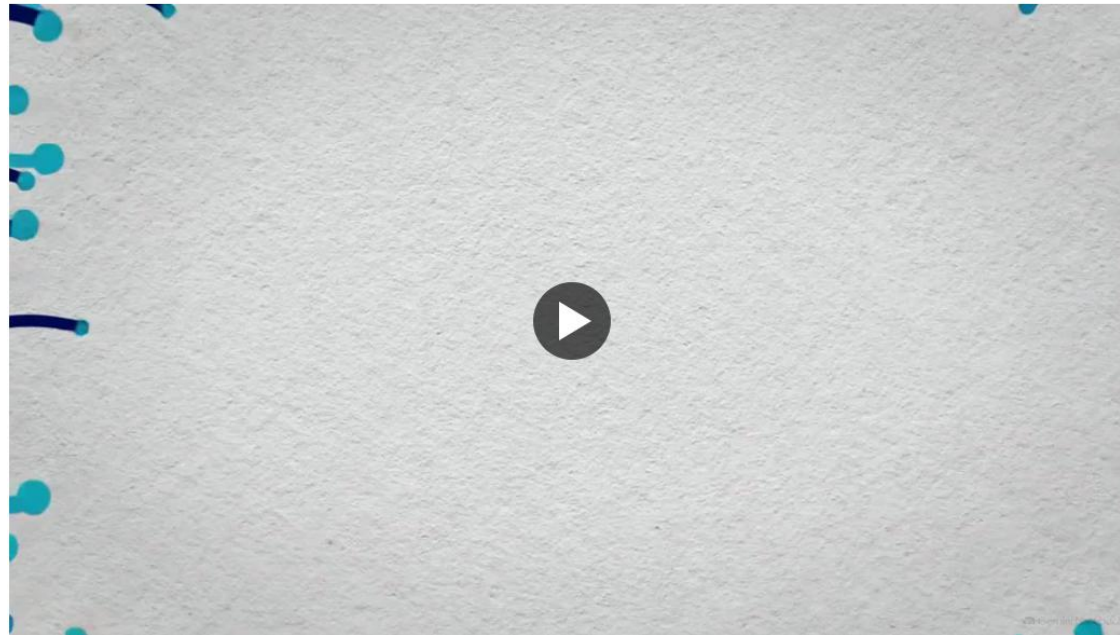
Exercise

30 minutes



Lab 5: Ingest data with a pipeline

Lab 5: Microsoft Fabric Dataflows Gen2 Lab



<https://www.linkedin.com/pulse/lakehouse-microsoft-fabric-labs-michael-lively-ejlx/>

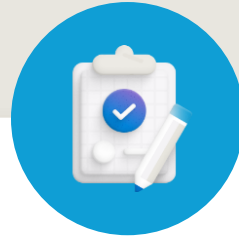
Thank you for attending this course

Here are some reminders:



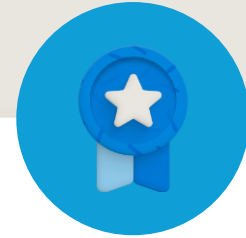
Celebrate your new skills

- Redeem your achievement
- Share with us and your network



Let us know how we did

- Give us your feedback
- Survey will be sent via email



Accelerate your success with Microsoft Credentials

- Explore additional resources to help prepare
- Schedule your credential

➔ Tag us social: **#AlwaysLearning**

Survey

<https://www.metricsthatmatter.com/url/u.aspx?A78F01505240987858>

